UNIVERSITY OF MINES AND TECHNOLOGY, TARKWA

FACULTY OF COMPUTING AND MATHEMATICAL SCIENCES

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

A PROJECT REPORT ENTITLED

DYNAMIC CONTEXT-AWARE TRIES FOR KNOWLEDGE GRAPH-CONSTRAINED LARGE LANGUAGE MODEL GENERATION

BY

BERNARD KIRK ADJANOR KATAMANSO

ERICA AMONOR

JOSEPH OSEI NYARKO

JESSICA AFUA ETORNAM NSAFOAH

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE AWARD OF THE DEGREE OF BACHELOR OF SCIENCE IN COMPUTER SCIENCE AND ENGINEERING

THESIS SUPERVISOR

.....

DR ERIC AFFUM

TARKWA, GHANA

APRIL 2026

DECLARATION

We, BERNARD KIRK ADJANOR KATAMANSO, ERICA AMONOR, JOSEPH OSEI NYARKO and JESSICA AFUA ETORNAM NSAFOAH, declare that this project work is our own original work. It is being submitted for the degree of Bachelor of Science in Computer Science and Engineering at the University of Mines and Technology (UMaT), Tarkwa. It has not been submitted, either in whole or in part, for any degree or examination at any other university.

Signature of Student: _____

BERNARD KIRK ADJANOR KATAMANSO (FOE.41.008.088.22)

Signature of Student: _____

AMONOR ERICA (FOE.41.008.030.22)

Signature of Student: _____

JOSEPH OSEI NYARKO (FOE.41.008.113.22)

Signature of Student: _____

JESSICA AFUA ETORNAM NSAFOAH (FOE.41.008.107.22)

_____ day of _____ (year) _____

ABSTRACT

Large language models are good at handling language, but they often produce confident outputs with incorrect facts. This issue, known as hallucination, is particularly problematic in knowledge graph question answering (KGQA), where finding the right answer often requires linking multiple reasoning steps through verified data. Methods like retrieval-augmented generation help tie responses to actual information. Graph-constrained decoding methods, such as Graph-Constrained Reasoning (GCR) and Decoding on Graphs (DoG), also limit what the model can produce based on known knowledge graph relationships. However, these constraints are set once and are not revisited. As the model creates an answer, the relevant context changes, but the rules on valid paths remain unchanged. Irrelevant paths through the knowledge graph stay available throughout.

This work introduces Dynamic Context-Aware Trie (DCA-Trie), which fixes that issue by continuously adjusting valid decoding paths as generation progresses. At each step, a semantic relevance scoring system based on pretrained sentence transformers assesses candidate knowledge graph paths against both the original question and what has already been generated. What qualifies as an acceptable next token is no longer fixed; it adapts to the changing interaction between graph structure, user intent, and the partial answer so far.

Two versions were developed. The first applies semantic filtering when the trie is created. The second updates those constraints step by step during decoding. To assess how restrictive or permissive the constraints are without linking the measure to whether the final answer is correct, we introduce the Semantic Irrelevance Ratio (SIR). Testing on WebQSP and ComplexWebQuestions shows that DCA-Trie produces fewer incorrect reasoning paths while remaining true to the structure of the underlying knowledge graph. Answer accuracy is on par with previous methods. A working prototype expands the evaluation beyond fixed benchmarks, demonstrating how the framework performs in live, open-ended question answering.

DEDICATION

Dedicated to our parents. For every sacrifice we didn't see and every bit of help we couldn't have asked for. Thank you for everything.

ACKNOWLEDGEMENTS

We extend our sincere gratitude to Dr. Eric Affum for his invaluable guidance, patience, and support throughout this project. His expertise in shaping our research direction has been instrumental in the completion of this dissertation.

We also wish to express our deep appreciation to our family, friends, and mentors. Their continuous encouragement, insight, and understanding provided the foundation we needed to see this work through to the end.

Finally, we acknowledge the broader academic community whose work has inspired and informed our research. This project stands on the shoulders of countless scholars, and we are grateful for their contributions to the field.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
BFS	Breadth-First Search
CoT	Chain-of-Thought
CWQ	ComplexWebQuestions
DCA-Trie	Dynamic Context-Aware Trie
DoG	Decoding on Graphs
FNR	False Negative Rate
FSM	Finite State Machine
GCD	Grammar-Constrained Decoding
GCR	Graph-Constrained Reasoning
GNN	Graph Neural Network
GPU	Graphics Processing Unit
KG	Knowledge Graph
KGQA	Knowledge Graph Question Answering
LLM	Large Language Model
NLP	Natural Language Processing
QA	Question Answering
RAG	Retrieval-Augmented Generation
RLHF	Reinforcement Learning from Human Feedback
SBERT	Sentence Bidirectional Encoder Representations from Transformers
SIR	Semantic Irrelevance Ratio
SQL	Structured Query Language
VRAM	Video Random Access Memory

CHAPTER 1

INTRODUCTION

1.1 Background

Large language models (LLMs) have advanced quickly over the past decade. Today, they handle language understanding, text generation, and multi-step reasoning quite well (Brown *et al.*, 2020; OpenAI, 2024). The GPT series (Brown *et al.*, 2020; OpenAI, 2024) and the Llama family (Touvron, Martin, and Stone, 2023; Grattafiori *et al.*, 2024) are strong examples of this progress. They learn patterns from large collections of text, which gives them broad knowledge in many areas. Still, this knowledge is stored in the model parameters rather than as clearly verified facts.

Despite recent improvements, LLMs still struggle with knowledge-heavy tasks because they sometimes generate information that is not backed by evidence, a problem known as hallucination. Ji *et al.* (2023) define hallucination as fluent output that either lacks evidence or is contradicted by reliable sources. Huang *et al.* (2025) further divide this into intrinsic hallucination, which goes against a given source, and extrinsic hallucination, which cannot be checked against any source. In practice, this is a real concern. Luo *et al.* (2025a) found that around one-third of retrieval-augmented cases included hallucinated reasoning steps, even when a knowledge graph was used. Recent studies also show that simply increasing model size will not fully solve this issue. Xu, Jain, and Kankanhalli (2025) show that it is not possible to guarantee zero hallucination for all inputs in computable learners. Thus, while training and prompting can help, achieving strict accuracy requires linking generation to external, verifiable structures.

The process behind this behavior is well understood. In autoregressive generation, each new token is chosen based on the probability $P(y_t \mid y_{<t}, x)$, where x is the input and $y_{<t}$ is the partial output (Brown *et al.*, 2020). This method keeps text locally fluent, but it does not check claims against outside sources while generating. If a wrong token appears at step t, it becomes part of the context for step $t + 1$ and can push the rest of the output off track (Huang *et al.*, 2025; Ji *et al.*, 2023).

This weakness is especially clear in knowledge graph question answering (KGQA). A knowledge graph stores facts as triplets such as (entity, relation, entity), for example, (Marie Curie, award received, Nobel Prize in Physics) (Bollacker *et al.*, 2008). Benchmarks such as WebQSP (Yih *et al.*, 2016) and ComplexWebQuestions (Talmor and Berant, 2018) are built on Freebase, which contains tens of millions of these triplets. Many questions require multi-step reasoning across several relations. For instance, answering "What is the nationality of the director of the film in which the Blue Hawaii actor starred?" requires following several linked facts in the correct order. An unconstrained LLM can still produce a fluent but incorrect answer because it relies on parametric memory instead of verifying each step (Lan *et al.*, 2022a; He *et al.*, 2021a).

Retrieval-augmented generation (RAG) is a widely used way to address this problem. It retrieves relevant passages or subgraphs and adds them to the prompt (Lewis *et al.*, 2020; Izacard and Grave, 2021). While this method is often useful, it does not provide strict control. The model may ignore retrieved evidence or generate information beyond it, because decoding remains unconstrained at the token level (Asai *et al.*, 2024; Agrawal *et al.*, 2024).

KG-constrained decoding addresses this limitation more directly. Instead of only adding KG information to context, it places a constraint oracle between model logits and token selection. At each step, the oracle applies a binary mask to block tokens that would break a valid KG path. As described by Geng *et al.* (2023), this is written as $\tilde{\alpha}_t = m_t \odot \alpha_t$, where α_t is the original logit vector and $m_t \in \{0, 1\}^{|V|}$ is the validity mask. Decoding then selects only from valid continuations, so facts outside the graph are structurally excluded during generation (Willard and Louf, 2023; Geng *et al.*, 2023).

Graph-Constrained Reasoning (GCR) is a strong example of this approach (Luo *et al.*, 2025a). It reaches 92.6 Hits@1 on WebQSP and 75.8 Hits@1 on CWQ with verified 100% structural faithfulness. GCR builds a KG-Trie, a prefix tree (Fredkin, 1960) containing all KG paths within L hops of question entities, and uses it during beam-search decoding. Because each generated token must match a valid trie prefix, outputs stay inside verified KG facts.

However, an important limitation remains. In GCR, the KG-Trie is built once before decoding using only graph structure and question entities. It does not adapt to question meaning, reasoning direction, or choices already made in $y_{<t}$. Formally, the valid-token set is

$W_{\text{val}} = f(G, E_q)$, which does not depend on q or $y_{<t}$. As a result, many valid but irrelevant paths can remain in the constraint set, especially for specific questions. The model must then filter these paths itself, bringing back reliance on the same parametric behavior that constrained decoding is meant to reduce (Li *et al.*, 2025; Luo *et al.*, 2025a).

Decoding on Graphs (DoG) (Li *et al.*, 2025) extends this by updating constraints step-by-step as the path grows. This makes search more aware of graph direction. However, question semantics are mainly used to rank beams, not to determine validity. Since validity is still based on connectivity, unrelated neighbors can remain in the valid set. DoG therefore only partially addresses permissiveness and also increases computational cost, because the structure is rebuilt repeatedly.

1.2 Problem Statement

Current KG-constrained methods, such as GCR (Luo *et al.*, 2025a) and DoG (Li *et al.*, 2025), define validity using graph structure and question entities. This ensures generated tokens remain on valid KG paths, but it does not fully account for question meaning or how that meaning should evolve during generation (Geng *et al.*, 2023; Willard and Louf, 2023).

This creates a gap between two goals. The first is structural validity, where each generated token follows a valid KG prefix. The second is contextual relevance, where the allowed set matches what the question needs at each step. Existing frameworks satisfy the first goal but not the second. As a result, their constraint sets are often too broad, allowing many irrelevant paths, and too static, showing little change as reasoning proceeds (Li *et al.*, 2025; Luo *et al.*, 2025a).

This project addresses a clear problem: current KG-constrained decoding frameworks do not update valid-token constraints based on the input question or partial output. Because of this, the LLM often chooses from many valid but irrelevant options at each step. This ongoing dependence on parametric knowledge can reintroduce the risk of hallucination that constrained decoding is meant to avoid. The issue becomes stronger in multi-hop settings, where the number of valid paths grows quickly with each hop (Lan *et al.*, 2022a; He *et al.*, 2021a; Talmor and Berant, 2018).

Recent work such as RouterKGQA (Yuan *et al.*, 2026) improves efficiency and faithfulness

by using model routing and semantic filtering at answer selection. DCA-Trie instead filters at the token-validity stage during decoding. This difference matters because filtering only after generation may help, but it does not prevent the model from considering irrelevant reasoning tokens during response formation. DCA-Trie therefore aims to reduce this search space directly. The framework also introduces the Semantic Irrelevance Ratio (SIR; Chapter 3), which measures oracle permissiveness independently of final accuracy.

To solve these limitations, this project introduces DCA-Trie by extending the validity function from $W_{\text{val}}(t) = f(G, E_q)$ to $W_{\text{val}}(t) = f(G, E_q, q, y_{<t})$. The improved oracle must meet three requirements: (i) preserve structural faithfulness, so every accepted token matches a valid KG path; (ii) reduce semantic irrelevance, shown by lower SIR than GCR and DoG; and (iii) maintain high answer recall, with false negatives on gold paths below 5% on WebQSP validation.

1.3 Objectives of Thesis

The aim of this project is to design, implement, and evaluate a Dynamic Context-Aware Trie (DCA-Trie) framework for knowledge graph-constrained large language model generation, in which the valid token constraint at each decoding step is conditioned jointly on the knowledge graph structure, input question semantics, and partially generated reasoning chain, improving answer accuracy on multi-hop KGQA benchmarks while maintaining 100% structural faithfulness to the underlying knowledge graph. Specific objectives are as follows:

i. to characterize the permissiveness problem in existing KG-constrained decoding frameworks by defining and measuring the Semantic Irrelevance Ratio (SIR) of GCR's static KG-Tries on the WebQSP and CWQ benchmarks, stratified by hop depth;
ii. to design a semantic relevance scoring mechanism using pretrained sentence transformer embeddings (Reimers and Gurevych, 2019) that can filter candidate KG paths by joint relevance to the input question and partial generation at inference time, without task-specific fine-tuning;
iii. to implement a static semantic filtering variant (DCA-Trie v1) that applies the relevance scorer at KG-Trie construction time, reducing trie size before generation begins while maintaining a false negative rate below 5% on gold answer paths on the WebQSP validation set;

iv. to implement a step-wise dynamic expansion variant (DCA-Trie v2) in which the trie is updated after each committed triplet using the relevance scorer conditioned jointly on the question and partial generation $y_{<t}$, directly operationalizing the $W_{\text{val}}(t) = f(G, E_q, q, y_{<t})$ formulation;

v. to evaluate DCA-Trie v1 and DCA-Trie v2 against the GCR baseline and an unconstrained chain-of-thought (CoT) baseline on WebQSP and CWQ, measuring Hits@1, F1, structural faithfulness, SIR, and average trie size, with results stratified by question hop depth; and

vi. to demonstrate how DCA-Trie works in practice by building an interactive prototype system, implemented as either a web-based or command-line QA interface for research demonstration, allowing users to enter complex natural language questions and receive fact-based reasoning chains with answers, showing the framework's value beyond benchmark evaluations.

1.4 Tools and Facilities Used

1.4.1 Facilities

The following resources were used in carrying out this project:

i. Google Colab Pro (A100, 40 GB VRAM) for all model inference and constrained decoding experiments.

ii. University library and online academic databases (ACM Digital Library, arXiv, Google Scholar, ACL Anthology) for literature review and citation verification.

iii. University internet infrastructure for cloud connectivity, dataset access, and retrieval of model checkpoints from the HuggingFace model hub.

iv. Shared team repository hosted on GitHub for version control, experiment logging, and reproducibility of all reported results.

1.4.2 Software Tools

Table 1.1 presents an evaluation of the software tools implemented in this project.

Table 1.1 Implemented Software Tools Evaluation.

Tool	Use in Project	Rationale
Python / HuggingFace Transformers	Model inference pipeline; loading and running GCR's Llama-3.1-8B checkpoint; integrating the relevance scorer	Provides reliable model loading, tokenization, and generation utilities, and enables direct use of the released GCR checkpoint.
GCR Framework	Primary baseline; KG-Trie construction, graph-constrained decoding, and result reproduction on WebQSP and CWQ	Provides a strong baseline and supports the trie-based constrained decoding mechanism that DCA-Trie extends.
Sentence-transformers / all-MiniLM-L6-v2	Semantic relevance scoring; computing cosine similarity between question-context and candidate KG path embeddings	Provides fast semantic embeddings without extra fine-tuning and supports efficient iterative decoding.
PyTorch	Deep learning framework underpinning all model inference and custom decoding logic	Supports implementation and testing of custom decoding logic and integrates smoothly with HuggingFace.
Freebase / WebQSP and CWQ Datasets	Knowledge graph source and KGQA benchmarks for evaluation	Provides standard datasets for this research area and enables direct comparison with prior work.
Google Colab Pro (A100, 40 GB)	GPU environment for all inference experiments; Llama-3.1-8B constrained decoding with beam search	Provides enough GPU memory for stable experiments and removes the need for local high-end hardware.
Visual Studio Code + Git / GitHub	Development environment and version control	Supports day-to-day coding, team collaboration, and version tracking.

1.5 Methods Used

The project employs the following methods:

i. Systematic review and critical analysis of the academic literature on constrained decoding, knowledge graph question answering, hallucination in LLMs, and retrieval-augmented generation.

ii. Formal gap analysis of the four foundational papers, GCR (Luo *et al.*, 2025a), DoG (Li *et al.*, 2025), GCD (Geng *et al.*, 2023), and Outlines (Willard and Louf, 2023), to identify the specific limitations that motivate the DCA-Trie design, including the permissiveness problem and the absence of generation-state feedback in the constraint oracle.

iii. Formal specification of the DCA-Trie framework, including definition of the Semantic Irrelevance Ratio metric, the relevance scoring mechanism using sentence transformer embeddings (Reimers and Gurevych, 2019), the static filtering procedure for v1, and the step-wise dynamic expansion rule for v2.

iv. Reproduction of the GCR baseline on the WebQSP benchmark using GCR's publicly released Llama-3.1-8B checkpoint to establish a verified starting point within 1–2% of published Hits@1.

v. Implementation of DCA-Trie v1 and v2 as extensions to GCR's inference pipeline, with relevance threshold selected on a 100-question held-out validation subset of WebQSP to minimize false negative rate on gold answer paths.

vi. Quantitative evaluation of all system configurations on WebQSP and CWQ using Hits@1, F1, faithfulness rate, SIR, and average trie size per step, stratified by question hop depth following the evaluation protocol of Lan *et al.* (2022a).

vii. Ablation studies examining the sensitivity of DCA-Trie's performance to the relevance threshold τ, the sentence encoder architecture, and beam size k, isolating the contribution of each mechanism to the overall result.

viii. Qualitative error analysis identifying failure cases in which DCA-Trie's semantic filter erroneously excludes a gold answer path, supporting honest characterization of the framework's limitations and providing concrete directions for future work.

1.6 Scope of Work

This project designs and evaluates a Dynamic Context-Aware Trie framework for KG-constrained LLM decoding, with the specific aim of addressing the permissiveness problem identified in GCR and DoG. The framework is developed and evaluated in the context of multi-hop KGQA over Freebase-grounded benchmarks (WebQSP and CWQ), which are the standard evaluation settings in the KG-constrained decoding literature. All experiments use GCR's publicly released Llama-3.1-8B checkpoint; no new model training or fine-tuning is performed. The semantic relevance scorer uses the all-MiniLM-L6-v2 sentence transformer without domain-specific adaptation. We emphasize that all experiments use GCR's publicly released Llama-3.1-8B checkpoint (Luo *et al.*, 2025a), which includes their fine-tuned KG-specialized parameters. No additional model training or fine-tuning is performed in this work; our contribution is strictly the constraint oracle architecture and semantic filtering mechanism.

The scope is explicitly bounded as follows. Generalization of DCA-Trie to knowledge graphs other than Freebase, or to non-KGQA tasks such as entity linking or structured information extraction, is identified as future work and is not evaluated in this dissertation. Formal proofs of the faithfulness guarantee under dynamic pruning are beyond scope; empirical measurement of the gold answer false negative rate is provided as evidence. The confidence-conditioned constraint tightening mechanism, in which constraint granularity is modulated by the LLM's token-level entropy, is described as a theoretical extension but is not implemented. Production-scale deployment, latency optimization for online serving, and integration with proprietary enterprise knowledge graphs are engineering extensions beyond the research scope of this project.

The primary contribution of this work is the DCA-Trie framework and the Semantic Irrelevance Ratio metric. Secondary contributions are the empirical characterization of constraint permissiveness in existing frameworks and the ablation analysis isolating the contribution of semantic filtering from dynamic expansion. We plan to share our code, experimental setups, and curated evaluation datasets so others can reproduce our results and build on this work.

1.7 Organization of Work

Chapter 1 introduces the study by outlining the main problem, project goals, and the planned DCA-Trie solution. This sets up the motivation, scope, and direction for the research. Chapter 2 reviews the literature, covering earlier work on hallucination in LLMs, KGQA, constrained decoding, and related frameworks. It also points out key gaps in current methods and explains why a context-aware constraint mechanism is needed. Chapter 3 covers the problem and methodology, describing the improved constraint setup, research questions, evaluation criteria, and how DCA-Trie v1 and v2 are used in the GCR pipeline (Luo *et al.*, 2025a). Chapter 4 presents the results and discussion, including quantitative evaluations on WebQSP and CWQ, analysis of faithfulness and semantic irrelevance, and findings from ablation and error analysis. Chapter 5 wraps up the dissertation by summarizing the main contributions, discussing practical implications, and suggesting directions for future research.

CHAPTER 2

LITERATURE REVIEW

While large language models (LLMs) are good at many tasks, their use of autoregressive generation makes them likely to produce fluent but incorrect information (Ji *et al.*, 2023). For knowledge-intensive question answering, this unreliability is a major roadblock: the ability to verify reasoning against external facts is essential. To tackle this issue, recent studies have focused on grounding models in structured external facts through knowledge graphs. Still, keeping a model strictly aligned with these graphs during generation, without losing its natural flow, is a difficult challenge.

This literature review looks at the current research on these challenges and examines how knowledge graph constraints might help ensure factual accuracy during multi-hop reasoning. Instead of immediately discussing the proposed solution, this chapter outlines the connections between language model hallucination, multi-step validation, and structured grounding. By assessing current strategies to reduce errors, from improved prompting to fixed decoding, this chapter shows where soft-grounding methods do not meet expectations, highlighting the specific technical gaps this project addresses.

2.1 Large Language Models

To understand how knowledge graph constraints can be used with language models, we first need to grasp how these models generate language. Large language models (LLMs) are the main systems powering today's natural language processing. This section describes the structure and generation features of LLMs. It focuses on the autoregressive process that leads to their smooth reasoning abilities and their risk of producing false information.

2.1.1 Transformer Architecture

Modern LLMs are built upon the Transformer architecture, introduced by Vaswani *et al.* (2017). At its core, the Transformer relies on a scaled dot-product self-attention mechanism that allows the model to dynamically weigh the importance of different tokens in an input sequence regardless of their positional distance. Formally, for a set of queries Q, keys K, and

values V, the attention output is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.1}$$

where d_k represents the dimensionality of the keys. This marked a significant change from earlier recurrent architectures. It allowed for highly parallel training and the capturing of long-range dependencies. Initially, the framework suggested an encoder-decoder setup for translation tasks. However, the field has mostly moved toward decoder-only models for text generation. In a decoder-only model, self-attention is causally masked. This means that when the model computes the representation for a specific token, it can only focus on previous tokens and not future ones. This strictly left-to-right processing fits well with the task of predicting the next word in a sequence. The Llama-3.1-8B model (Dubey *et al.*, 2024) is a prominent example of this decoder-only, causally-masked transformer design, and has been adopted as the backbone in recent KG-constrained decoding work (Luo *et al.*, 2025a).

2.1.2 Autoregressive Generation

The text production process in these models is fundamentally autoregressive. Given a vocabulary $\mathcal{V}$, an input sequence x, and previously generated output tokens $y_{<t}$, the model computes a probability distribution over the entire vocabulary for the next token: $P(y_t|y_{<t}, x)$. Since the generation condition includes all previously generated tokens, each new word becomes part of the context for the next word. Decoding strategies, such as beam search and nucleus sampling (Holtzman *et al.*, 2020), determine how a specific token is chosen from this calculated distribution. Importantly, because this distribution is always computed over the full vocabulary, any mathematically possible token has a positive, non-zero chance of being generated at any step. This autoregressive nature is what creates fluent, coherent text. However, since each step relies on prior outputs without an external verifier, it also directly causes hallucination.

2.1.3 Large-Scale Pretraining

Large language models emerged through a fundamental shift in scale: current models are trained on vastly larger datasets using billions more parameters than earlier transformers. Foundational models are pretrained on vast amounts of unlabelled text using a simple

self-supervised goal, usually predicting the next token. The GPT series (Brown *et al.* 2020; OpenAI 2024) and the open-weight Llama series (Touvron, Martin, and Stone 2023; Dubey *et al.* 2024) show that scaling this simple task leads to powerful downstream capabilities, including few-shot learning and broad world knowledge. During pretraining, the model internalizes patterns, facts, and logical structures found in its training data. However, this memory is fundamentally static and statistical. While increasing the size of models improves their fluency, syntax, and ability to recall common facts, it does not address the underlying generation process. The model still lacks a way to interact with dynamic external facts or ensure that its statistical outputs reflect verified truths in the real world.

2.1.4 Instruction Following and Chain-of-Thought

To better align these pretrained base models with user intent, they go through post-training processes like instruction tuning and reinforcement learning from human feedback (RLHF). This alignment lets models engage in dialogue and follow complex instructions. One of the key abilities of aligned LLMs is multi-step reasoning, often prompted through Chain-of-Thought (CoT; Bosma *et al.* 2022). As surveyed by Huang and Chang (2023), CoT prompting encourages the model to produce intermediate reasoning steps before reaching a final answer, effectively breaking down complex problems. While CoT greatly enhances performance on logic and reasoning tasks, it suffers from the same autoregressive limitation: each intermediate reasoning step is generated using only the model's internal statistical distributions. So, while CoT helps structure a language model's reasoning, it cannot logically connect those outputs to verifiable sources. This ongoing lack of factual grounding shows the need for stricter interventions, such as knowledge graph-constrained decoding.

2.2 Hallucination in Large Language Models

2.2.1 Definition and Taxonomy

Hallucination in natural language generation refers to the production of content that is fluent and internally coherent yet factually unsupported by or in outright contradiction with verifiable sources (Ji *et al.*, 2023). Huang *et al.* (2023) refine this definition into a two-part taxonomy. *Intrinsic hallucination* happens when the model output directly contradicts material present in the provided source context. *Extrinsic hallucination* occurs when the output contains claims

that cannot be verified against any external source, even if it does not directly contradict a specific reference.

Both types are documented at non-trivial rates in deployed systems. Ji *et al.* (2023) survey over 30 works across summarization, dialogue, and question answering, finding hallucination rates ranging from 15% to over 60% depending on task type and measurement methodology. In the specific context of knowledge graph question answering, Luo *et al.* (2025a) report that retrieval-augmented reasoning approaches produce hallucinated reasoning steps in approximately one-third of cases even when provided direct access to a supporting knowledge graph, underscoring that the problem is not simply one of insufficient external knowledge.

2.2.2 Structural Causes: The Autoregressive Generation Mechanism

The primary cause of hallucination is the autoregressive generation process found in all modern large language models. At each generation step, the model computes a probability distribution over its vocabulary based on the input and all previously generated tokens. Three structural aspects of this mechanism make hallucination an ongoing risk rather than just a rare failure.

First, the generation distribution is based entirely on learned parameters, with no part that corresponds to factual verification against an external source. The model has no mechanism to "check" a candidate token against a knowledge base before selecting it. Second, errors propagate forward: because each step is conditioned on all prior output, a hallucinated token at step t becomes part of the conditioning context for step $t+1$, producing fluent and internally consistent continuations of an already-incorrect chain. Third, the distribution is defined over the full vocabulary at every step, meaning that any token has a positive chance of being selected at any point, regardless of factual correctness (Brown *et al.*, 2020; Huang *et al.*, 2023; Ji *et al.*, 2023).

2.2.3 The Scaling Paradox

A common argument against the severity of the hallucination issue is that large models should eventually achieve enough factual reliability. However, evidence does not support this view. Lin, Hilton, and Evans (2022) introduce TruthfulQA, a benchmark for questions that often produce plausible but false responses. Their main finding is that larger models do not perform

better than smaller models in terms of truthfulness and in some categories show *inverse scaling*. In other words, larger models can do worse because they reproduce widespread but incorrect beliefs from their training data more effectively.

Perez *et al.* (2023) identify a compounding issue in models trained with reinforcement learning from human feedback: these models learn to generate confident-sounding responses that align with apparent user expectations rather than verifiable facts, separating confidence from accuracy. Kandpal *et al.* (2023) show that while larger models memorize more training-data facts and improve on frequently-seen entity combinations, they do not improve on compositional questions requiring combinations of facts not seen during training. This precisely mirrors the structure of multi-hop knowledge graph questions. Mallen *et al.* (2023) confirm significant hallucination rates for GPT-4 on questions involving tail entities, regardless of model size. Together, these findings show that hallucination is a structural feature of the autoregressive mechanism, not a scaling issue with a simple solution.

2.3 Multi-Step Reasoning: Chain-of-Thought and Its Limits

2.3.1 Chain-of-Thought Prompting

Chain-of-thought (CoT) prompting, introduced by Bosma *et al.* (2022), is the dominant paradigm for eliciting multi-step reasoning from large language models. By including worked examples in the prompt that show intermediate reasoning steps, the model learns to decompose complex problems into a sequence of simpler sub-problems before producing a final answer. Bosma *et al.* (2022) demonstrate substantial accuracy improvements on arithmetic, commonsense, and multi-hop question answering benchmarks, with the capability emerging at approximately 100 billion parameters. Zero-shot chain-of-thought (Kojima *et al.*, 2022) achieves similar benefits without worked examples by adding the phrase "Let's think step by step" to the prompt, demonstrating that the underlying capability is latent in sufficiently large models and requires only elicitation.

Despite these improvements, CoT has a fundamental structural limitation: each intermediate step is generated by the same autoregressive mechanism described in the previous section. An incorrect intermediate step propagates forward as conditioning context, and no mechanism verifies that any step corresponds to a fact in an external knowledge source. A CoT chain can

be entirely fluent and internally coherent while containing one or more steps that are factually fabricated.

2.3.2 Extensions: Self-Consistency and Tree-of-Thought

Two major extensions of chain-of-thought tackle its reliability issues without fixing its structural faithfulness problem. Self-consistency (Wang *et al.*, 2023c) samples several independent CoT chains and chooses the final answer based on majority vote, minimizing individual chain errors. This method significantly improves accuracy on benchmarks like GSM8K and multi-hop QA by reducing sensitivity to individual hallucinated chains. However, it does not guarantee that any single chain is factually accurate. A majority of independently hallucinated chains can still produce an incorrect majority-vote answer, especially on deep multi-hop questions where error rates increase.

Tree-of-Thought (Cao *et al.*, 2023a) treats generation as a search through a tree of partial reasoning sequences. It uses the language model both as a generator and as an evaluator to trim unpromising branches. This approach enhances performance on tasks that need backtracking and exploration. However, the key limitation persists: pruning decisions are made by the model itself, relying on its own parameters as the only evaluator, without any outside verification of the factual accuracy of candidate reasoning steps.

2.3.3 The Faithfulness Ceiling of Prompting-Based Approaches

The main limitation of all prompting-based methods, such as few-shot, chain-of-thought, self-consistency, tree-of-thought, and their combinations, is that they modify the *input* to the language model without modifying the *generation mechanism*. Since decoding happens over the full vocabulary, each token has a strictly positive probability at every step. True structural faithfulness means ensuring that each generated token matches a verified external fact. To achieve this, invalid tokens must be given exactly zero probability. This can only happen by directly changing the decoding algorithm. This is the purpose of constrained decoding methods (Geng *et al.*, 2023; Willard and Louf, 2023).

2.4 Knowledge Graphs and Knowledge Graph Question Answering

2.4.1 Knowledge Graphs

A knowledge graph is a structured database of verified facts, represented as a set of directed, labelled triplets (e, r, e') where $e, e' \in \mathcal{E}$ are entities and $r \in \mathcal{R}$ is a relation type. Each triplet encodes a single human-verified fact: for example, (*Marie Curie*, *award_received*, *Nobel_Prize_Physics*) or (*Blue_Hawaii*, *film.film.featured_film_location*, *Hawaii*). Figure 2.1 illustrates a simplified fragment of such a structure, demonstrating how these triplets interlink to form a broader network. Freebase (Bollacker *et al.*, 2008), the knowledge graph underlying the evaluation benchmarks used in this work, encodes tens of millions of such triplets across geography, history, entertainment, and science, and has been widely adopted as the standard resource for KGQA research (Yih *et al.*, 2016; Talmor and Berant, 2018).

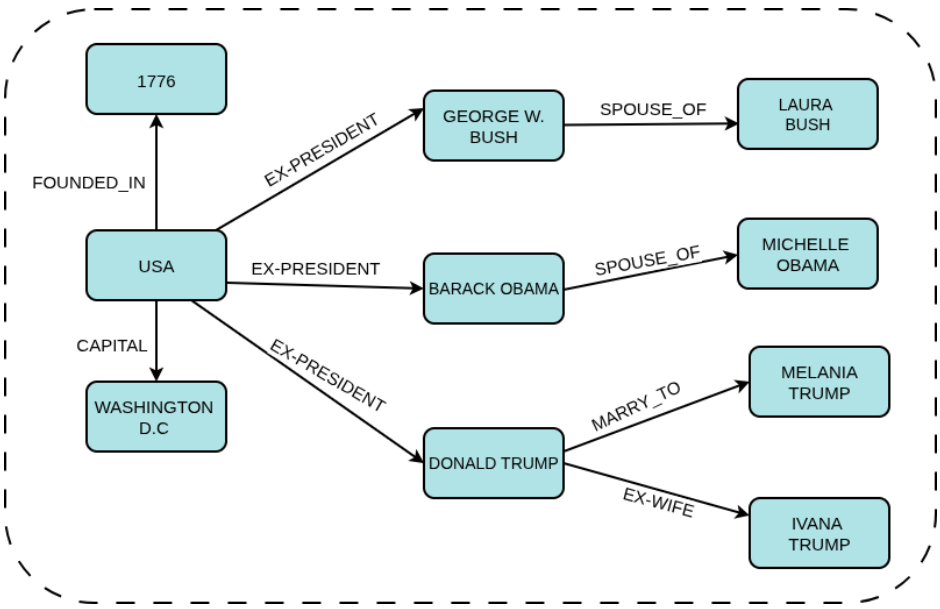

Figure 2.1 A simplified example of a knowledge graph fragment.

The critical distinction between a knowledge graph and an unstructured text corpus is that every triplet in the KG has been explicitly verified and is machine-readable in a deterministic, unambiguous form. This property makes KGs suitable as sources for hard constraints on generation: a candidate token can be verified against the KG via an exact lookup, rather than via probabilistic semantic matching against raw text. A multi-hop reasoning path of length L through the KG is a sequence $(e_0, r_1, e_1, \ldots, r_L, e_L)$ in which each consecutive triplet (e_{i-1}, r_i, e_i) is a verified member of $\mathcal{T}$ (Lan *et al.*, 2022b).

2.4.2 Knowledge Graph Question Answering

Knowledge graph question answering (KGQA) requires a system to identify the answer entity e_L to a natural language question q by reasoning through a chain of verified KG triplets. The task is formally defined as: given question q with associated question entities $\mathcal{E}_q \subseteq \mathcal{E}$ identified by entity linking, identify the terminal entity of the correct reasoning path in $\mathcal{G}$ (Lan *et al.*, 2022b).

Early KGQA approaches relied on either semantic parsing (converting questions to structured SPARQL queries) or information retrieval over KG subgraphs (Lan *et al.*, 2022b). More recent approaches use large language models as retrievers, agents iteratively querying the KG, or generators conditioned on retrieved subgraph context (Jiang *et al.*, 2023a; Jin *et al.*, 2024; Luo *et al.*, 2024). The move toward LLM-based KGQA has improved fluency and coverage, but it has also brought back the risk of hallucination that structured methods avoided. LLM-generated reasoning chains are not guaranteed to match confirmed KG paths.

2.4.3 Evaluation Benchmarks

Two benchmark datasets provide the evaluation framework for KGQA research. WebQSP (Yih *et al.*, 2016) comes from the WebQuestions dataset. It contains 4,737 questions based on Freebase, with reasoning depths of one to two hops. These questions include SPARQL queries and are evaluated using Hits@1 and F1. ComplexWebQuestions (CWQ; Talmor and Berant 2018) builds on WebQSP by transforming questions through conjunction, composition, and comparative operations, resulting in about 34,000 questions that require up to four reasoning hops. WebQSP illustrates a case where the valid paths are relatively few. In contrast, CWQ shows a case with many valid paths, making it harder to find the correct path among structurally valid options.

2.4.4 Multi-Hop Complexity

The complexity of multi-hop KGQA rises sharply with hop depth. For an entity with an average out-degree d in the KG, the number of valid paths of length L from the question entities increases as $|\mathcal{E}_q| \times d^L$. For well-connected Freebase entities where d is around 20 and a question may have up to three question entities, a three-hop question can generate up to 24,000 structurally valid paths, but only one leads to the correct answer (He *et al.*, 2021b;

Lan *et al.*, 2022b). This rapid growth has real implications for any method that needs to distinguish between candidate paths. At each step, the model must choose the correct next token from a vast array of valid yet incorrect options, relying on its parametric knowledge to do so.

2.5 Constrained Decoding for Faithful Text Generation

Constrained decoding restricts generation to a predefined valid set at each step by directly intervening in the decoding mechanism. Unlike prompting, which changes the input to the LLM, constrained decoding alters the token selection process itself. Invalid tokens receive zero probability before sampling, making their generation structurally impossible.

2.5.1 Logit Masking

The main mechanism of constrained decoding is logit masking, formally described in Geng *et al.* (2023) and Willard and Louf (2023). This technique works by computing a binary mask over the vocabulary at each generation step, setting the log-probability of invalid tokens to $-\infty$ before the softmax, thereby forcing their selection probability to exactly zero. This ensures that invalid tokens are structurally impossible to generate, rather than merely discouraged. The following subsections examine two concrete instantiations of logit masking: first for enforcing syntactic and formal correctness, then for enforcing knowledge graph path validity.

2.5.2 Grammar-Constrained Decoding

Geng *et al.* (2023) introduce Grammar-Constrained Decoding (GCD), which constrains generation to strings conforming to a context-free grammar, using an Earley parser as the oracle. The valid next-token set at each step is the set of tokens that extend the current partial string as a valid grammar prefix. GCD additionally introduces input-dependent grammars, in which the grammar is parameterized by the input question, representing a partial step toward context-sensitive constraints. Willard and Louf (2023) address the computational cost of constrained decoding with an efficient finite-state machine formulation. By precomputing an index from automaton states to valid vocabulary subsets, the valid token set can be retrieved in amortized constant time at each generation step, making constrained decoding practical at inference speed.

The shared limitation of these format-constraint approaches is that they enforce structural validity of the output format (such as grammatical correctness or schema conformance) without enforcing factual correctness. A grammatically valid output can contain entirely hallucinated facts. The transition from format constraints to factual constraints requires a constraint oracle that verifies tokens against a knowledge source rather than against a grammar.

2.5.3 Knowledge Graph-Constrained Decoding

Applying constrained decoding to knowledge graph faithfulness is a more recent development. The central idea is to use the KG itself as the constraint oracle. At each generation step, only tokens that correspond to valid continuations of a KG reasoning path are allowed. This method guarantees structural faithfulness, meaning every generated token matches a verified KG triplet, making hallucination of KG facts structurally impossible.

Graph-Constrained Reasoning (GCR) (Luo *et al.*, 2025a) is the leading work in this area. GCR builds a KG-Trie (a prefix tree of all KG paths within L hops of the question entities, created by breadth-first search before generation begins) and uses it as the constraint oracle during beam-search decoding. The trie is queried at each step to provide valid next tokens. GCR achieves 92.6 Hits@1 on WebQSP and 75.8 on CWQ with verified 100% structural faithfulness, using a Llama-3.1-8B backbone. One important feature is that the trie is created once before generation and remains unchanged: the valid token set is fixed no matter what has been generated.

Decoding on Graphs (DoG) (Li *et al.*, 2025) presents step-wise trie expansion as a partial solution to GCR's static oracle. DoG defines *well-formed chains* as sequences of KG triplets where each triplet exists in the KG, and each entity is connected to the previous triplet or is a question entity. It enforces this rule through step-wise expansion. After locking in an entity, the constraint trie is expanded with all KG neighbors of that entity. This makes the oracle topologically dynamic, growing as reasoning moves through the graph. DoG achieves 90.99 Hits@1 on WebQSP and 76.08 on CWQ with 100% structural faithfulness, and it is training-free, needing no fine-tuning. However, the expansion rule includes all KG neighbors of each visited entity, regardless of their relevance to the question or current reasoning direction. Additionally, constructing the trie step-by-step compromises the efficiency of the precomputed constraint structures.

RouterKGQA (Yuan *et al.*, 2026) takes a different approach by routing between a specialized KG-reasoning model and a general LLM based on question complexity. It uses semantic filtering with SBERT embeddings (specifically `nomic-embed-text-v1`) to select candidate answers after generation, achieving competitive accuracy while lowering inference costs. RouterKGQA's semantic scoring operates at the answer selection level, rather than at the token constraint level. Paths are created first and filtered afterward, instead of being constrained during generation.

2.5.4 Beam Search and Trie-Based Constraints

To understand how knowledge graph constraints are applied during generation, it is important to look at the two main search algorithms and data structures involved: beam search and the prefix trie.

Beam Search. First introduced in the HARPY speech recognition system (Lowerre, 1976), beam search is a heuristic search algorithm used in natural language processing to decode sequences from probabilistic models. Unlike greedy decoding, which only picks the single most probable token at each step and risks creating sub-optimal global sequences, beam search keeps a set of the k most promising partial hypotheses (or "beams") at all times. At each generation step, the algorithm expands all k current hypotheses by scoring their potential continuations, ranks the resulting sequences based on their total log-probability, and reduces the search space back to the top k candidates (Holtzman *et al.*, 2020). This method reduces the chance of early errors multiplying while still being computationally viable. Thus, it is the primary decoding strategy for tasks that need structured and coherent output.

The Prefix Trie. A trie, originating from the concept of "trie memory" (retrieval) introduced by Fredkin (1960), is a deterministic tree-based data structure used to store a changing set of sequences, usually strings. In a trie, no single node holds a complete string. Instead, a node's position in the tree indicates the exact prefix it represents. The branches from any node show the set of valid next characters or tokens. In constrained decoding, a pre-computed trie acts as a highly efficient constraint oracle. Given the current sequence prefix, navigating the trie to the correct node yields the specific subset of valid next tokens in constant time, $O(1)$, concerning vocabulary size. This operation is significantly quicker than continuous dynamic parser evaluations, making trie lookups the most scalable option for applying firm

factual constraints at inference speed (Willard and Louf, 2023).

The KG-Trie. For knowledge graph-constrained generation, models like Graph-Constrained Reasoning (GCR) use a specific adaptation known as the KG-Trie (Luo *et al.*, 2025a). A KG-Trie serializes structurally verified multi-hop paths from the knowledge graph into string formats and maps them across the language model's precise token vocabulary. Rather than nodes representing single alphabetical characters, each edge in the KG-Trie represents a specific valid token in the LLM's vast vocabulary space.

When beam search is merged with a KG-Trie oracle as visualized in Figure 2.2, the decoding process transforms into a strictly bounded traversal of the prefix tree.

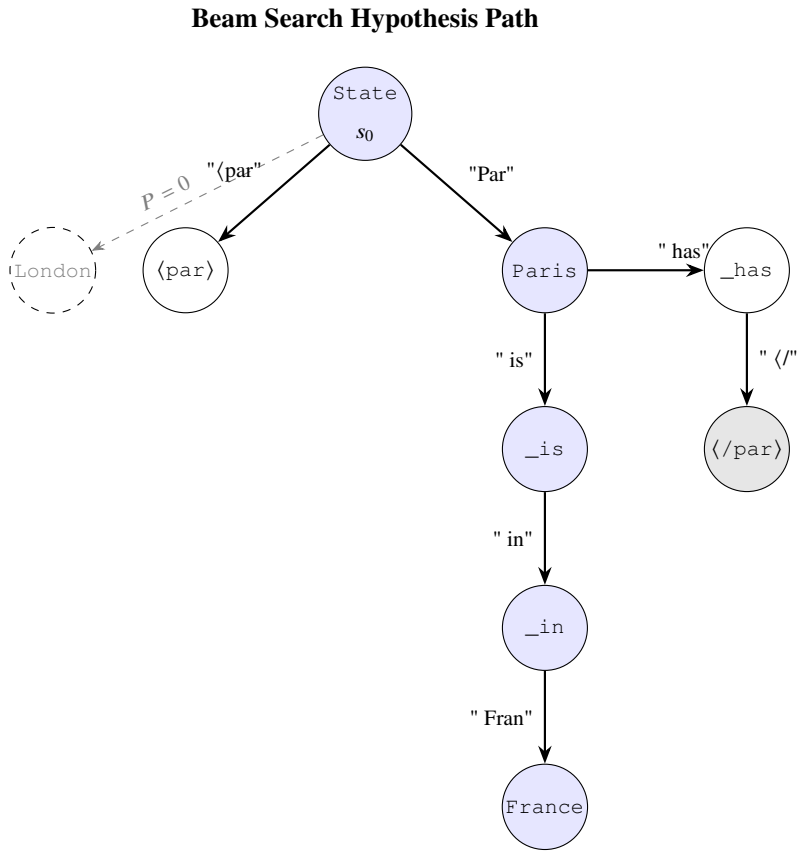

Figure 2.2 Visualization of KG-Trie Constrained Generation.

At each autoregressive step, the current beam hypotheses map to specific states in the KG-Trie. The language model calculates a probability distribution over its entire vocabulary, but a mathematical mask forces the probability of any candidate token not validly present as a child node in the trie to exactly zero ($-\infty$ prior to softmax normalization). The beam search then advances strictly down the permissible branches, functionally restricting the LLM's

autoregressive generation to a set of pre-verified deterministic paths while still allowing the model to express internal preference (via log-probabilities) among those factually valid chains.

2.6 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG), introduced by Lewis *et al.* (2020) in 2020, is the leading approach for grounding LLM outputs in external knowledge. A dense retriever picks relevant passages from an external source based on embedding similarity. A reader LLM then generates the answer based on the retrieved context and the question. Fusion-in-Decoder, proposed by Izacard and Grave (2021) in 2021, enhances this by encoding multiple retrieved passages separately and merging them in the decoder, which improves coverage.

2.6.1 KG-Augmented Retrieval

Several studies enhance RAG with knowledge graph (KG)-structured context. He *et al.* (2021b) retrieve KG subgraphs and use attention on retrieved facts to aid multi-hop reasoning. Yasunaga *et al.* (2021) combine retrieved text passages and KG neighborhoods through graph neural networks. Shi *et al.* (2021) retrieve KG paths as natural language context strings, providing path-shaped input to the reader. G-Retriever, developed by He *et al.* (2024) in 2024, applies RAG specifically to textual graph understanding, retrieving graph-structured context for question-answering tasks.

2.6.2 RAG Variants

Recent RAG variants focus on the timing and accuracy of retrieval. Self-RAG (Asai *et al.*, 2024) adds reflection tokens that allow the model to decide when retrieval is necessary and whether retrieved passages support the generated claim. This improves retrieval precision and citation accuracy. FLARE (Jiang *et al.*, 2023b) uses the model's next-token prediction confidence as a trigger for retrieval, obtaining new context when confidence drops below a certain threshold. IRCoT (Trivedi *et al.*, 2023) mixes retrieval with chain-of-thought steps, fetching relevant context at each reasoning step, significantly improving multi-hop accuracy. GraphRAG (Edge *et al.*, 2024) builds community-structured knowledge graphs from document collections and retrieves community summaries for complex relational queries. This approach uses graph structure to boost coverage.

2.6.3 The Structural Limitation of Soft Grounding

Despite their empirical improvements, all RAG approaches share a structural limitation: retrieved context is a soft influence on generation. The retriever determines what the LLM sees; it does not determine what the LLM is permitted to generate. Because the LLM's generation mechanism remains unchanged (the distribution over the full vocabulary is computed autoregressively), there exists a nonzero probability of generating any token at any step, including tokens that correspond to no verified KG fact. Structural faithfulness with probability one, which guarantees that every generated token corresponds to a verified fact, cannot be achieved under any architecture that modifies only the input context without modifying the token selection mechanism (Luo *et al.*, 2025a; Geng *et al.*, 2023).

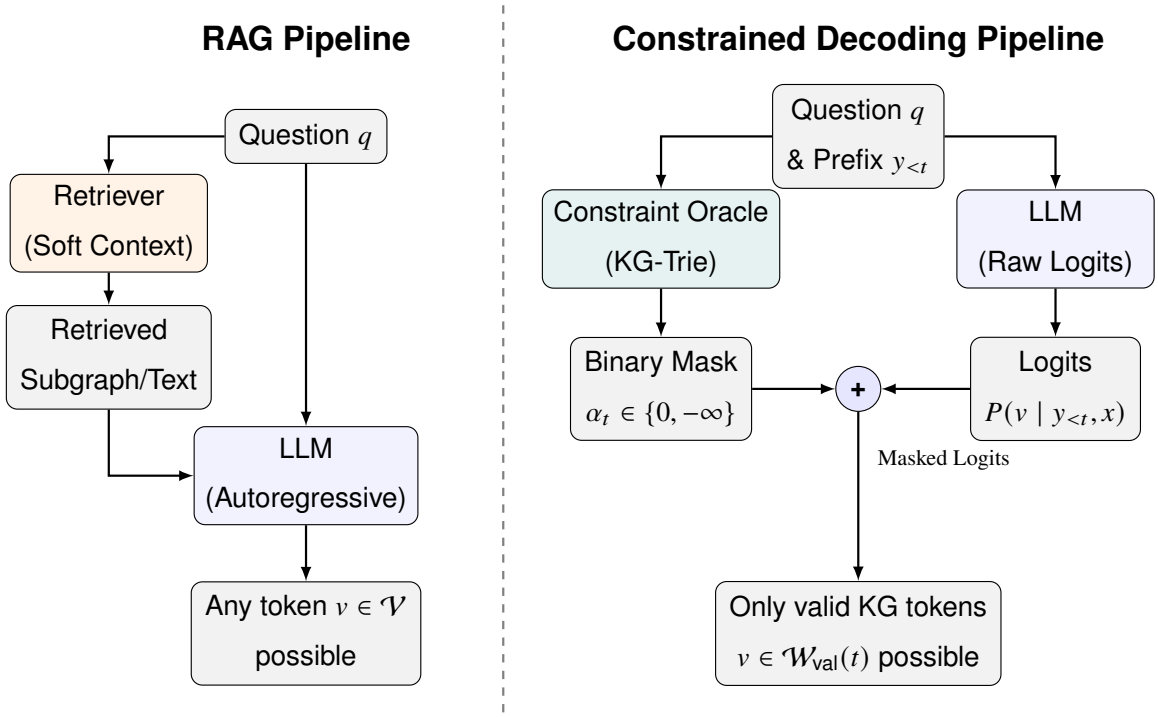

Figure 2.3 Comparison of standard RAG (left) and KG-constrained decoding (right).

2.7 Semantic Relevance Scoring with Sentence Transformers

2.7.1 Sentence Transformer Embeddings

Sentence-BERT (Reimers and Gurevych, 2019) introduced a class of sentence encoders trained to produce dense vector representations in which semantically similar sentences cluster together and semantically dissimilar sentences are pushed apart. The SBERT architecture

applies mean pooling over token representations from a BERT-like transformer encoder, producing a fixed-length embedding. Training uses a siamese network on labelled sentence pairs with a classification objective over entailment, neutral, and contradiction labels. The resulting embedding space supports cosine similarity as a reliable proxy for semantic alignment between sentences, without requiring task-specific fine-tuning at inference time.

2.7.2 Applications in Retrieval and Graph Reasoning

Sentence transformer similarity has been applied extensively in dense retrieval settings. Karpukhin *et al.* (2020) demonstrate that a bi-encoder architecture trained on question-passage pairs produces embeddings that outperform sparse retrieval (BM25) on open-domain QA. Xiong *et al.* (2021) refine bi-encoder training, improving coverage of relevant passages. In the KGQA context, Shi *et al.* (2021) and Saxena, Chakrabarti, and Talukdar (2022) apply semantic similarity to rank candidate KG paths for retrieval, selecting high-scoring paths as soft context for the reader LLM. These approaches use cosine similarity as a soft ranking signal over candidate paths, selecting the top-k for context provision.

2.7.3 Encoder Architecture Trade-offs

Choosing a sentence encoder involves balancing quality and latency, which is particularly important in step-wise constrained decoding, where the encoder is used at each generation step. Larger encoders like all-mpnet-base-v2 (110M parameters) and BGE-M3 (Chen *et al.*, 2024) (568M parameters) perform better on semantic textual similarity benchmarks but significantly increase inference latency. Lexical methods like BM25 work quickly but cannot capture semantic similarity between paraphrase-equivalent paths that lack surface tokens. The all-MiniLM-L6-v2 model (Reimers and Gurevych, 2019), with 22M parameters and about 80ms per-query CPU latency, strikes a good balance between semantic quality and inference speed for iterative applications.

2.8 Related Works

This section reviews the broader field of knowledge-augmented reasoning across two complementary research trajectories: (1) *soft-grounding* approaches, which use external knowledge to inform or filter generation without modifying the decoding mechanism, and (2) *hard-constraint* approaches, which restrict generation to verified facts by modifying the token

selection process itself. The survey progresses from early structured methods → modern LLM-based soft approaches → semantic relevance techniques → hard-constraint frameworks → concurrent work, establishing how these lines of research have developed largely in parallel without integration at the constraint-oracle level. This progression illuminates the specific technical gaps that motivate the proposed approach.

2.8.1 Semantic Parsing and Early Structured KGQA

Before large language models became mainstream, traditional methods mainly addressed KGQA through semantic parsing or structured representation learning. Two key approaches characterize this period: parsing questions into executable KG queries like SPARQL and aligning question and KG representations into a shared space for reasoning.

In the first group, early semantic parsing relied on fixed question templates to create queries, which required deep domain knowledge (Berant *et al.*, 2013; Reddy, Lapata, and Steedman, 2014; Bast and Haussmann, 2015). More recent methods use deep learning to automatically generate these logical forms (Yih *et al.*, 2015; Lan and Jiang, 2020). While these symbolic methods offer high interpretability and precise reasoning, they face major challenges in practice. They need expensive ground-truth logical forms for training, which are hard to obtain, and their performance drops sharply when the KG has missing links. Additionally, model-generated queries often cannot be executed due to small syntax or semantic errors with the underlying graph schema, resulting in no answers and lacking fallback options (Yu *et al.*, 2023; Luo *et al.*, 2025a).

To ease the demand for strict logical forms, the second group of methods integrates text and graph representations into a continuous shared space for reasoning. Some methods match question representations to KG facts (Miller *et al.*, 2016; Xu *et al.*, 2019) or KG structural representations (Zhang *et al.*, 2018; Sen, Mavadia, and Saffari, 2023). Notable examples include GraftNet (Sun *et al.*, 2018), PullNet (Sun *et al.*, 2019), and NSM (He *et al.*, 2021c), which enhance reasoning through deep graph-based approaches. Incomplete graphs have also been explored, where missing links are inferred using KG embeddings (Saxena, Tripathi, and Talukdar, 2020). While these structured techniques avoid hallucination by operating over confirmed data, their expressiveness is limited compared to the generative flexibility and complex multi-step reasoning that modern LLMs provide.

2.8.2 Retrieval-Based KGQA

Earlier KGQA methods relied on structured retrieval from the knowledge graph instead of LLM generation. These methods prevent LLM hallucination by avoiding freeform generative models and working within the structured limits of the KG. The following works cover the main techniques in this area, including semantic parsing, embedding-based retrieval, graph neural network propagation, and path-level supervision.

Yih *et al.* (2016) created the WebQSP benchmark and introduced a semantic parsing method that converts natural language questions into SPARQL queries to be executed against Freebase. This method offers exact symbolic faithfulness by limiting answer derivation to formally verified SPARQL execution. However, it has a key limitation: the parsing step is fragile. Small changes in question phrasing can lead to incorrect parses, resulting in structurally valid but semantically wrong SPARQL queries. Additionally, this method requires extensive annotation of SPARQL queries for training, which is costly and hard to scale for complex compositional questions.

Talmor and Berant (2018) introduced ComplexWebQuestions and a related compositional semantic parsing method that breaks down complex questions into simpler sub-questions before generating SPARQL. This decomposition technique expands the coverage of semantic parsing for compositional queries, but it does not fix the fragility issue. Errors during decomposition can accumulate across sub-questions, causing the final SPARQL query to be structurally valid but answer a subtly different question than the one asked.

He *et al.* (2021b) suggested a multi-hop KGQA method that learns intermediate supervision signals to guide path selection through the KG. They showed that explicit intermediate supervision at each hop significantly enhances performance at deeper reasoning levels. By training the model to predict correct intermediate entities, this approach helps ease the combinatorial difficulties of multi-hop searches. Yet, its limitation is that these intermediate supervision signals come from annotated answer paths, which are expensive to gather and might not be available for all benchmarks or KGs. Furthermore, this method is limited to a fixed maximum hop depth and does not adjust its reasoning depth based on the complexity of the question.

Yasunaga *et al.* (2021) introduced QA-GNN, which combines retrieved text passages and KG neighborhood entities in a joint graph neural network. This network propagates information between passage token nodes and KG entity nodes. By reasoning over both types of information, QA-GNN improves performance on questions that require a combination of textual and structured evidence. However, the joint graph mixes verified KG facts with unverified textual claims, meaning noisy or incorrect text passages can distort entity scores derived from the KG structure. The approach also needs to build a joint graph at inference time for each question, leading to significant preprocessing overhead.

Shi *et al.* (2021) retrieve KG reasoning paths by calculating semantic similarity between the question and natural language descriptions of candidate paths. They provide the top-k highest-scoring paths as context strings to a reader LLM. This retrieval-plus-reading setup separates the structured retrieval step from the natural language generation step. The limitation is that the ranking function relies on soft cosine similarity over surface-form path descriptions, which might give high scores to paths that are lexically similar to the question but semantically incorrect. The reader LLM receives paths as plain text, lacking a way to verify that the provided paths are actual KG edges.

Saxena, Chakrabarti, and Talukdar (2022) used entity and relation embeddings to score and select multi-hop KG paths without using an LLM generator. They learned representations from KG structure with embedding objectives, and computed multi-hop path scores by combining relation embeddings. This method stays entirely within the structured KG and avoids generative hallucination. However, embedding-based path scoring struggles to capture the full semantic complexity of natural language questions. Compositional questions involving negation, comparison, or conjunction are hard to encode accurately as relations, leading to a limit on the types of complex questions it can handle.

Lan *et al.* (2022b) conducted a thorough survey of KGQA methods, analyzing the trade-offs between semantic parsing, embedding-based, and retrieval-based techniques across various benchmarks. They found that retrieval-based approaches often have difficulty with tail entities and rare relation types that are not well-represented in training data. They also noted that no single retrieval method excels across all question types and hop depths. This survey supports the shift toward LLM-based methods as more adaptable alternatives.

Sun *et al.* (2019) developed PullNet, which iteratively gathers relevant KG facts and text passages into a dynamically built evidence graph. It performs inference over this graph to answer multi-hop questions. The iterative method allows PullNet to gather evidence at various depths without setting a hop depth in advance. However, its limitation is that the evidence graph combines KG and text sources without assessing their reliability, and the iterative process introduces latency that increases with question complexity.

Das *et al.* (2018) presented "Go for a Walk and Arrive at the Answer," an end-to-end path-finding approach that trains a policy to navigate the KG from a question entity to the answer entity. Reinforcement learning uses the correctness of answers as a reward to train the policy. This method directly models the reasoning process as graph traversal and yields interpretable paths. Its limitation is that reinforcement learning with sparse correctness rewards is quite unstable during training, needing careful reward shaping. The policy also does not generalize well to question types not adequately represented in the training data.

Zhang *et al.* (2022) proposed subgraph-based reasoning that focuses on extracting a relevant subgraph from the full KG. They perform message passing over this subgraph before selecting the answer entity. By limiting reasoning to a local subgraph, this method significantly reduces both the search space and the cost of inference. The subgraph boundary is determined by how close entities are to the question entity, which creates a ceiling on recall. Answer entities that are topologically farther from the question entity in the KG might be left out of the subgraph, even if they are the correct answer. This issue is closely tied to the hop-depth assumptions built into subgraph extraction methods.

These retrieval-based methods prevent LLM hallucination by working within the structured KG rather than generating freely. However, they share a limitation in expressiveness. They are usually restricted to fixed path structures, fixed hop depths, or fixed retrieval limits and do not take advantage of the flexible multi-step reasoning capabilities of modern LLMs. The shift to LLM-based methods in more recent research improved the naturalness and coverage for complex compositional questions, but this came at the expense of the structural faithfulness guarantees provided by retrieval-based methods.

2.8.3 Knowledge Graph-Enhanced LLM Reasoning

More recent advances focus on the joint use of KGs and LLMs for KGQA, leveraging the sophisticated capabilities of LLMs for enhanced reasoning while mitigating hallucinations and knowledge gaps. A foundational observation is that pure prompting-based approaches (such as Chain-of-Thought (CoT) (Bosma *et al.*, 2022), Plan-and-solve (Wang *et al.*, 2023b), DecomP (Khot *et al.*, 2023), Self-consistency (Wang *et al.*, 2023d), and Tree-of-Thought (ToT) (Cao *et al.*, 2023b)) achieve multi-step reasoning but not structural faithfulness to external facts. Fine-tuning approaches extending to reinforcement learning (e.g., OpenAI o1) (Hoffmann *et al.*, 2022) improve reasoning quality but still lack hard guarantees against hallucination. Consequently, the field has shifted toward integrating KG access directly into the generation process.

Agent-based and exploratory approaches. Several methods treat LLMs as agents that iteratively query KGs, such as ReACT (Yao *et al.*, 2023), StructGPT (Jiang *et al.*, 2023a), and ToG (Sun *et al.*, 2024). These methods prompt the LLM to explore relevant facts hop-by-hop using beam search on the KG, combining the LLM's reasoning capacity with the KG's structural grounding.**Path retrieval for intermediate supervision.** Others use explicit retrieval of intermediate facts to guide generation. Methods like KD-CoT (Wang *et al.*, 2023a) and RR (He *et al.*, 2022) retrieve facts to guide generative plans. Jin *et al.* (2024) propose Graph Chain-of-Thought (GraphCoT) to explicitly retrieve graph neighborhood information at every intermediate CoT step, making the connection between reasoning and KG structure explicit.

Specialized encoders and fusion architectures. More sophisticated approaches use graph neural networks and specialized encoders to bridge text and graph modalities. GNN-RAG (Mavromatis and Karypis, 2025) and GFM-RAG (Luo *et al.*, 2025b) adopt graph neural networks to capture relational structure. Luo *et al.* (2024) introduce Reasoning on Graphs (RoG), a planning-retrieval-reasoning framework that retrieves reasoning paths for faithful reasoning. StructLM (Zhuang *et al.*, 2024) fine-tunes models over large structured datasets, while DECAF (Yu *et al.*, 2023) merges semantic parsing and LLM reasoning. These approaches achieve higher empirical performance but often depend on instruction-following or require fine-tuning, creating barriers to practical deployment.

Methods emphasizing complete graph exploration. In contrast to methods that retrieve subgraphs, recent work such as Decoding on Graphs (DoG) (Li *et al.*, 2025) and Tree-of-Traversals (Markowitz *et al.*, 2024) reason over the full accessible KG without filtering, enabling complete coverage at the cost of increased search space.

Shared limitation: KG knowledge as soft guidance. Despite architectural diversity, most reviewed methods treat KG knowledge as a suggestion rather than a strict constraint. For example, UniKGQA (Jiang *et al.*, 2022) unifies retrieval and reasoning through shared representations but introduces entanglement between the two failure modes. Methods using embedding-based approaches (OpenKE (Han *et al.*, 2018)) internalize KG knowledge as parametric memory, losing hard faithfulness guarantees. Subgraph-based methods (Zhang *et al.*, 2022; Wang *et al.*, 2021; Sun *et al.*, 2019; Lin, Yang, and Zhang, 2022; Mavromatis and Karypis, 2022) provide KG context but cannot prevent the LLM from generating beyond it. Even with perfect retrieval, the absence of hard structural constraints means the LLM can still ignore or contradict provided facts during generation.

A direct measurement of this limitation comes from Luo *et al.* (2025a), who find that even the most effective KG-enhanced methods produce hallucinated reasoning steps in approximately one-third of cases. This empirically establishes that soft KG grounding has a systematic performance ceiling.

2.8.4 Semantic Similarity in Graph Reasoning

Several studies use semantic similarity signals to guide reasoning over knowledge graphs. Shi *et al.* (2021) rank candidate KG paths based on the semantic similarity between the question and path descriptions, retrieving high-scoring paths as evidence. Saxena, Chakrabarti, and Talukdar (2022) learn embeddings for KG entities and relations to support similarity-based multi-hop path selection. He *et al.* (2021b) use attention-guided traversal over retrieved KG subgraphs, with attention weights indicating how relevant each subgraph region is to the question. PathRetriever (Asai *et al.*, 2020) learns to find reasoning paths from evidence graphs by scoring paths using a recurrent model with their node sequences.

These studies show that semantic similarity can effectively tell apart relevant and irrelevant KG paths in a retrieval setting. The main difference in architecture is that semantic similarity

serves as a soft ranking signal. Candidate paths receive scores, and the highest-scoring ones are presented to the LLM as context or selected among the generated candidates. No existing work treats semantic similarity as a hard binary criterion that determines which tokens the LLM can generate at each step. The difference between soft retrieval signals and hard generation constraints is significant. Soft signals influence what the LLM sees, while hard constraints dictate what the LLM can produce.

While semantic similarity provides soft ranking signals to guide retrieval and filtering, a complementary research direction pursues hard constraints that structurally restrict generation at each decoding step. Rather than choosing among many candidate outputs after generation, hard constraint approaches eliminate invalid options before they are produced. This distinction, between soft guidance and hard structural restriction, is fundamental and leads to qualitatively different performance guarantees and interpretability.

2.8.5 Format-Constrained and Entity-Constrained Generation

A separate line of research tackles factual accuracy by imposing strict limits on what a generative model can produce, regardless of the input context. Unlike retrieval-based approaches that limit the information available to the model, these studies change the generation mechanism itself. The following cluster follows the development of this approach from early lexical constraints to grammar-based and automaton-based frameworks.

De Cao *et al.* (2021) introduced autoregressive entity retrieval (GENRE), which limits generation to a specific set of entity names using a trie created from all known entity surface forms. By restricting beam search to valid trie paths, GENRE ensures that each generated token sequence corresponds to a real entity in a fixed vocabulary.

Geng *et al.* (2023) generalise entity-level trie constraints to arbitrary context-free grammar constraints in Grammar-Constrained Decoding (GCD). GCD uses an Earley parser as the constraint oracle, admitting only tokens that extend the current partial string as a valid grammar prefix. GCD additionally introduces input-dependent grammars parameterised by the input question, allowing the valid token set to depend on question content. GCD establishes that grammar constraints are computationally tractable and empirically improve structured output quality across multiple tasks. The limitation is that a context-free grammar

can enforce syntactic or format correctness but cannot encode factual membership in a knowledge graph: a string may conform perfectly to a grammar specification while containing entirely hallucinated entities or relations.

Willard and Louf (2023) address the computational cost of grammar-constrained decoding with an efficient finite-state machine (FSM) formulation. By precomputing an index from FSM states to valid vocabulary subsets, the valid token set can be retrieved in amortised constant time at each generation step, making constrained decoding practical at inference speed for production systems. This formulation reduces the per-step constraint overhead from the Earley parsing cost (which is cubic in input length) to effectively $O(1)$ amortised lookup. The limitation remains the same as GCD: the FSM encodes structural or format correctness, not factual correctness. An FSM-constrained system can still generate structurally valid outputs that are factually incorrect with respect to any external knowledge source.

Deutsch, Upadhyay, and Roth (2019) propose a general algorithm for constrained sequential inference applicable across any structured prediction task, framing constraint enforcement as inference in a product of finite-state automata. This framework unifies a broad range of constraint types under a common algorithmic schema and demonstrates that hard constraint enforcement is compatible with a variety of sequence models. Its limitation for KGQA is that the constraints it enforces are defined over output string structure rather than over external knowledge: the automata encode properties of the output form, not membership in a KG edge set.

Poesia *et al.* (2022) introduce Synchromesh, a constrained generation system for program synthesis that uses a completion engine to enforce syntactic validity of generated code. By grounding constraint enforcement in a language-specific parser, Synchromesh ensures that all generated programs are syntactically valid. This work establishes that constraint oracles can be derived from formal language specifications beyond context-free grammars. Applied to KGQA, an analogous approach would enforce syntactic validity of SPARQL queries; however, a syntactically valid SPARQL query may still reference entities or relations absent from the target KG, producing an empty result set rather than a faithfully grounded answer.

Scholak, Schucher, and Bahdanau (2021) present PICARD, a method for parsing SQL queries with constrained autoregressive inference. PICARD uses an incremental parser to

reject tokens that would make the partial output unparseable or semantically invalid with respect to a database schema, thereby constraining generation to schema-valid SQL. PICARD demonstrates that schema-grounded constraints substantially reduce invalid query generation in text-to-SQL tasks. The limitation for KG settings is that SQL schema constraints enforce structural consistency with a fixed schema, not factual path traversal through a dynamic relational graph. A schema-valid query may still produce an incorrect answer if the underlying reasoning is unsound.

Anderson *et al.* (2017) introduce Guided Open Vocabulary Image Captioning (GOVIC), an early demonstration of constrained beam search for lexically-constrained generation in image captioning, enforcing inclusion of specified words in the output. While the domain differs from KGQA, this work establishes constrained beam search as a practical mechanism for enforcing lexical constraints during generation and demonstrates the feasibility of hard inclusion constraints. Its limitation is that lexical constraints enforce presence of specific surface forms without enforcing their factual correctness or logical coherence with the rest of the generated output.

Hokamp and Liu (2017) introduce constrained decoding via a grid beam search algorithm that enforces specified lexical constraints in neural machine translation. Grid beam search maintains separate hypothesis sets for each subset of satisfied constraints and merges them to produce outputs satisfying all constraints. This algorithm demonstrates that complex constraint sets can be enforced in polynomial time during beam search. The limitation is that the constraint set must be specified in advance and is fixed throughout generation; there is no mechanism for the constraint oracle to consult an external knowledge source at each step to determine which tokens are valid continuations.

Lu *et al.* (2021) propose NeuroLogic Decoding, a decoding algorithm that enforces propositional logic constraints over lexical items during generation, extending the class of enforceable constraints beyond simple inclusion to disjunctions, conjunctions, and negations. NeuroLogic demonstrates that logically complex constraints can be enforced efficiently during beam search without prohibitive computational overhead. For KGQA applications, the limitation is that propositional logic constraints over lexical items still operate at the surface-form level: a NeuroLogic constraint can require that a specific entity name appears in the output, but cannot verify that this entity is connected to the preceding entity by a verified KG edge.

Zhang *et al.* (2023) analyse the computational tractability of constrained decoding under various constraint classes, establishing theoretical bounds on the complexity of constraint enforcement as a function of grammar type and vocabulary size. Their analysis confirms that finite-state constraints admit efficient enforcement via precomputed indices, while context-sensitive constraints require per-step evaluation that may be prohibitively slow. This theoretical result directly motivates the design choice in KG-constrained systems of representing valid paths as tries (finite-state structures) rather than as arbitrary graph reachability queries.

These works collectively establish that hard constraints on generation are computationally tractable and empirically beneficial across multiple domains. Their shared limitation for KGQA is that they enforce output *format* rather than *factual content*: constraining generation to grammatically valid strings, schema-valid queries, or lexically specified surface forms does not ensure that the generated strings correspond to verified KG paths. The extension from format constraints to KG path constraints requires a constraint oracle that encodes the relational structure of the knowledge graph itself, mapping valid token sequences not to grammar derivations but to traversable edges in a specific, dynamic knowledge base.

2.8.6 Hallucination Mitigation in Knowledge-Intensive Tasks

Beyond the constrained decoding approach, various methods aim to reduce hallucination in knowledge-intensive tasks. Retrieval-augmented methods, such as RAG (Lewis *et al.*, 2020), Self-RAG (Asai *et al.*, 2024), FLARE (Jiang *et al.*, 2023b), IRCoT (Trivedi *et al.*, 2023), and GraphRAG (Edge *et al.*, 2024), address this issue by providing relevant external context. This improves factual coverage without changing the generation mechanism. Wang *et al.* (2023c) enhance consistency by majority-voting across multiple Chain-of-Thought (CoT) paths. Huang *et al.* (2024) investigate whether LLMs can self-correct reasoning errors, finding that self-correction is usually ineffective without external feedback. This reinforces the idea that internal knowledge cannot reliably detect its own mistakes.

Post-hoc verification methods (Nakano *et al.*, 2022; Gao *et al.*, 2023) check generated claims against external sources after generation and either revise or flag unsupported statements. While these methods effectively identify errors, they require a separate verification step and do not prevent hallucination during generation. Agrawal *et al.* (2024) systematically analyze failure modes in RAG systems and find that even when relevant context is provided, LLMs

often produce statements not backed by the retrieved evidence.

2.8.7 Concurrent Work

RouterKGQA (Yuan *et al.*, 2026) is the work most closely related to this research in terms of shared components. RouterKGQA directs KGQA queries between a specialized KG reasoning model and a general LLM based on question complexity. It applies SBERT-based semantic similarity (using the nomic-embed-text-v1 encoder) to filter candidate reasoning paths. It achieves strong performance with lower inference costs, averaging about 1.15 LLM calls per question.

The key difference in architecture lies in when semantic scoring is applied. RouterKGQA scores complete reasoning paths after they are generated, using similarity to filter among candidates afterward. This method does not alter the token selection process during generation. The LLM generates freely, and semantic similarity is applied post-hoc to select among the outputs. In contrast, this thesis uses semantic similarity as a hard admission criterion at the token-constraint level. In this setup, similarity determines which tokens the LLM can generate at each decoding step, rather than just which completed outputs are kept later. This represents a qualitatively different architectural change that RouterKGQA does not address. RouterKGQA also does not provide a metric for constraint permissiveness and does not measure how much its candidate sets contain valid but semantically irrelevant paths.

2.9 Synthesis: Three Unresolved Gaps

The review shows that two different research paths, constrained decoding for structural faithfulness and semantic similarity for relevance-guided reasoning, have developed mainly in parallel without being combined at the constraint-oracle level. Constrained decoding methods (Luo *et al.*, 2025a; Li *et al.*, 2025) achieve structural faithfulness but create their constraint oracles without considering question semantics or the context of generation. This leads to valid token sets that include many irrelevant paths. Semantic similarity methods (Shi *et al.*, 2021; Saxena, Chakrabarti, and Talukdar, 2022; Yuan *et al.*, 2026) improve relevance but act as soft signals that affect retrieval or selection after the fact, without changing the generation process.

Three specific gaps emerge from this review.

Gap 1: The Permissiveness Problem. Current KG-constrained decoding frameworks build their constraint oracles based on graph structure and question entities only. They do not consider question semantics or the current generation state. In the case of multi-hop questions, this results in valid token sets with thousands of structurally correct but semantically irrelevant paths at each generation step. The LLM has to sort through these paths using its own knowledge, which reintroduces the risk of hallucination that constrained decoding aimed to eliminate (Luo *et al.*, 2025a; Li *et al.*, 2025).

Gap 2: The Static-Dynamic Efficiency Tradeoff. GCR's precomputed static trie is efficient in computing but lacks semantic awareness and remains the same at every generation step. DoG's step-wise topological expansion adds some dynamism but requires reconstructing the trie at every step and does not include semantic filtering (Li *et al.*, 2025; Willard and Louf, 2023). No current framework achieves semantic responsiveness, where the valid set narrows progressively as reasoning moves in a specific direction, while also maintaining the efficiency of precomputed constraint structures.

Gap 3: The Absence of a Constraint Quality Metric. No existing work offers a metric that measures constraint permissiveness separately from the accuracy of downstream answers. Without this metric, it is not possible to track progress on reducing irrelevant paths in the valid set or to compare frameworks based on constraint quality without factoring in model quality. These three gaps highlight a key challenge at the intersection of the two research paths discussed in this chapter. This challenge involves integrating semantic relevance scoring directly into the constraint oracle and conditioning the valid token set based on the knowledge graph, question semantics, and the evolving state of generation.

Table 2.1 summarizes the key works reviewed in this chapter, categorizing them by their approach, whether they provide structural faithfulness guarantees, and whether they incorporate semantic conditioning into their constraint mechanisms.

Table 2.1 Summary of Key Related Works

Work	**Approach**	**Structural faithfulness**	**Semantic conditioning**
Luo *et al.* (2025a)	Static KG-Trie decoding	Yes (100%)	No
Li *et al.* (2025)	Step-wise topological expansion	Yes (100%)	No
Yuan *et al.* (2026)	Routing + post-hoc filtering	Probabilistic	Yes (answer-level)
Luo *et al.* (2024)	Reasoning path planning	No	Partial
Jiang *et al.* (2023a)	LLM over structured data	No	Partial
Jin *et al.* (2024)	Graph-augmented CoT	No	Partial
Jiang *et al.* (2022)	Unified retrieval and reasoning	No	Partial
Wang *et al.* (2021)	KG-pretrained language model	No	Partial
Mavromatis and Karypis (2022)	Iterative reasoning revision	No	Partial
He *et al.* (2021b)	KG subgraph attention	No	Partial
Yasunaga *et al.* (2021)	Text-KG graph neural network	No	Partial
Das *et al.* (2018)	RL-based KG path traversal	No	Soft
Saxena, Chakrabarti, and Talukdar (2022)	Embedding-based path selection	No	Soft
Shi *et al.* (2021)	Path retrieval by similarity	No	Soft (retrieval)
Asai *et al.* (2024)	Retrieval with reflection	No	Soft
Edge *et al.* (2024)	Community-structured retrieval	No	Soft
Geng *et al.* (2023)	Grammar-constrained decoding	Format only	No
Willard and Louf (2023)	FSM-indexed constrained decoding	Format only	No
De Cao *et al.* (2021)	Entity trie-constrained decoding	Entity names only	No
Scholak, Schucher, and Bahdanau (2021)	Schema-constrained SQL parsing	Schema only	No
Lu *et al.* (2021)	Logic-constrained decoding	Format only	No

These three gaps collectively identify an open problem at the intersection of the two research trajectories reviewed in this chapter: integrating semantic relevance scoring directly into the constraint oracle, conditioning the valid token set jointly on the knowledge graph, the question semantics, and the evolving generation state.

CHAPTER 3

METHODOLOGY

3.1 Introduction

This chapter outlines the method for the Dynamic Context-Aware Trie (DCA-Trie), a system aimed at solving the permissiveness problem in graph-constrained decoding. To check if DCA-Trie effectively balances structural faithfulness with contextual relevance, this chapter introduces a diagnostic metric called the Semantic Irrelevance Ratio (SIR). After defining the main problem and the SIR metric, the discussion explains the underlying semantic scoring method and the design of both DCA-Trie variants. Finally, it sets up the baseline systems, the evaluation protocol, and the specific ablation strategies used to isolate the framework's effects, establishing the limits for the experimental results discussed in Chapter 4.

3.2 Formal Problem Specification

3.2.1 Restating the Core Limitation of Existing Frameworks

As established in Chapter 2, current graph-constrained frameworks (notably GCR (Luo *et al.*, 2025a) and DoG (Li *et al.*, 2025)) rely on a deterministic oracle that looks only at the fixed graph $\mathcal{G}$ and the starting entities $\mathcal{E}_q$. Regardless of whether the system builds this oracle before decoding or expands it on the fly, the set of valid tokens at any step t remains locked to structural logic:

$$\mathcal{W}_{\text{val}}^{\text{GCR}}(t) = f(\mathcal{G}, \mathcal{E}_q) \quad \forall\, t \tag{3.1}$$

By ignoring both the semantic intent of the question q and the reasoning trajectory already captured in the prefix $y_{<t}$, this formulation treats every structurally reachable path as equally plausible. In dense, multi-hop Freebase environments, this blindness causes catastrophic search space explosion. The framework might admit over 8,000 paths at depth 3, even though only a single path logically answers the query.

The core issue is that these systems conflate two separate requirements. *Structural validity* ensures the LLM only generates tokens that correspond to real graph edges, which current

models do well. *Contextual relevance* ensures the LLM only generates tokens that logically connect to the specific question at hand, which current models fail to do. This gap between structural compliance and semantic usefulness is the *permissiveness problem*.

3.2.2 The Upgraded Oracle Specification

DCA-Trie addresses this problem by defining an upgraded constraint oracle that combines structural validity with semantic context. The oracle conditions on both the input question and the evolving generation prefix:

$$\mathcal{W}_{\text{val}}^{\text{DCA}}(t) = f(\mathcal{G},\, \mathcal{E}_q,\, q,\, y_{<t}) \tag{3.2}$$

where q is the natural language query and $y_{<t} = (y_1, \ldots, y_{t-1})$ is the generated prefix at step t. To be methodologically sound, the upgraded oracle must satisfy three conditions aligned with the project objectives in Chapter 1:

1. **Structural Faithfulness (Condition F):** Every candidate token $v \in \mathcal{W}_{\text{val}}^{\text{DCA}}(t)$ must correspond to a valid prefix of a path $p \in \mathcal{G}$ at every step t. This preserves the 100% structural faithfulness guarantee achieved by GCR and DoG and prevents ungrounded tokens from entering the valid set.
2. **Semantic Relevance Reduction (Condition R):** The Semantic Irrelevance Ratio (SIR) under DCA-Trie must be lower than the corresponding SIR under GCR when averaged over the evaluation corpus. Using SIR as defined in Section 3.4:

$$\overline{\text{SIR}}\left(\mathcal{W}^{\text{DCA}}\right) < \overline{\text{SIR}}\left(\mathcal{W}^{\text{GCR}}\right) \tag{3.3}$$

3. **Recall Preservation (Condition P):** Semantic pruning must not remove gold paths excessively. Operationally, the false negative rate (FNR) on the WebQSP validation split must remain below 5%:

$$\text{FNR} = \frac{|\{q \in Q_{\text{val}} : p_q^* \notin \mathcal{W}_{\text{val}}^{\text{DCA}}(t)\}|}{|Q_{\text{val}}|} < 0.05 \tag{3.4}$$

   where p_q^* is the gold path for question q and Q_{val} is the held-out 100-question validation set.

3.3 System Architecture Overview

DCA-Trie focuses on the constraint oracle layer, so its architecture directly builds on the baseline pipeline from Graph-Constrained Reasoning (GCR) (Luo *et al.*, 2025a). This reuse helps isolate the experimental effects. By keeping the rest of the generation pipeline constant, the system ensures that any improvement in latency or search efficiency comes only from the new oracle mechanism. The four layers work as follows:

1. **Entity Linking Layer (Unmodified):** A named entity recognition tool first identifies the core query entities $\mathcal{E}_q$ from the raw text question q. These entities dictate where the downstream search algorithm can start exploring the knowledge graph.
2. **Constraint Oracle Layer (DCA-Trie Contribution):** This layer controls which tokens the LLM can generate. In GCR, this layer creates a static prefix tree (a KG-Trie) that includes every possible blind path reachable within L hops of the starting entities. DCA-Trie completely revamps this. Through either static pre-filtering (v1) or dynamic, step-by-step expansion (v2), the new oracle ensures that no structural edge is added to the trie unless it also shows semantic relevance to the question.
3. **Constrained Decoding Layer (Unmodified):** During autoregressive generation using beam search, this layer intercepts the LLM logit distribution and applies a binary mask retrieved from the oracle layer. By forcing the LLM to output only tokens that are structurally and semantically approved, it maintains 100% ground-truth faithfulness to the knowledge graph.
4. **Inductive Reasoning Layer (Unmodified):** After the constrained beam search produces K top reasoning paths, GPT-4o-mini analyses these paths at the same time to synthesize the final natural language answer.

Figure 3.1 shows the layout, isolating the constraint oracle layer to emphasize where the core semantic intervention takes place.

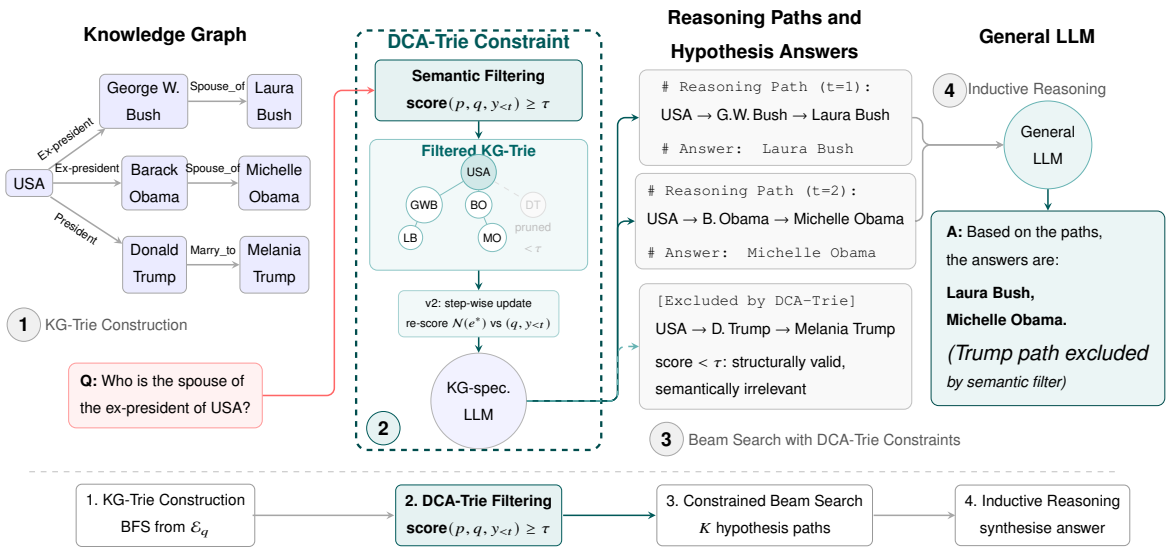

Figure 3.1 DCA-Trie System Architecture.

3.4 The Semantic Irrelevance Ratio: Definition and Measurement

3.4.1 Standard Metric Deficiencies

Current performance markers for knowledge graph decoding, such as Hits@1, F1, and structural faithfulness ratios, have a significant drawback: they focus on output instead of the process. While these metrics show whether a framework found the correct answer and stayed within the graph, they do not reveal how the framework reached that answer. A model may solve the correct gold path, but it could wander through a vast number of irrelevant paths that waste resources.

Therefore, confirming accuracy is not enough; we need to measure the constraint mechanism itself to ensure computational focus. To assess how lenient the oracle is, without considering the final output accuracy, this thesis presents a new diagnostic metric: the Semantic Irrelevance Ratio (SIR).

3.4.2 Formal Definition of SIR

Suppose $\mathcal{P}(q,t)$ represents all paths that the constraint oracle allows at decoding step t for question q. To calculate SIR, the system must first define irrelevance. An individual candidate path $p \in \mathcal{P}(q,t)$ is structurally valid, but it fails the semantic stringency test if its contextual

similarity drops below a hard baseline threshold τ_{ref}:

$$\text{irrelevant}(p, q, y_{<t}) = \mathbf{1}\left[\cos\big(E(p),\ E(\text{concat}(q, y_{<t}))\big) < \tau_{\text{ref}}\right] \tag{3.5}$$

where $E(\cdot)$ is the `all-MiniLM-L6-v2` sentence encoder, and $\tau_{\text{ref}} = 0.3$ is a fixed reference threshold. This reference threshold is intentionally non-tunable and separate from DCA-Trie's admission threshold τ.

The step-level SIR for question q at decoding step t is then defined as:

$$\text{SIR}(q, t) = \frac{\sum_{p \in \mathcal{P}(q,t)} \text{irrelevant}(p, q, y_{<t})}{|\mathcal{P}(q, t)|} \tag{3.6}$$

Corpus-level SIR is computed by averaging over all questions and all valid decoding steps up to depth L:

$$\overline{\text{SIR}} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{L_q} \sum_{t=1}^{L_q} \text{SIR}(q, t) \tag{3.7}$$

where L_q is the hop length of the generated reasoning chain for question q.

3.4.3 Properties and Interpretation

By definition, $\overline{\text{SIR}} \in [0, 1]$. A value near 1 indicates that most admitted paths are irrelevant, so the model still faces a large and noisy search space. A value near 0 indicates that the constraint set is tightly aligned with question semantics. Prior constrained frameworks such as GCR and DoG tend to show high SIR, especially at larger hop depths where path counts increase rapidly. DCA-Trie is designed to reduce SIR by filtering semantically weak paths while retaining structurally valid ones. In reporting, SIR is analyzed by hop depth to evaluate how this effect scales with reasoning complexity.

3.5 Semantic Relevance Scoring Mechanism

3.5.1 Scorer Architecture

The semantic relevance scorer implements the transition from a static oracle $f(\mathcal{G}, \mathcal{E}_q)$ to a context-aware oracle $f(\mathcal{G}, \mathcal{E}_q, q, y_{<t})$. As illustrated by the complete workflow in Figure 3.2,

it takes a candidate KG path p, the question q, and the partial generation $y_{<t}$, and returns a relevance score in $[0, 1]$.

A path $p = (e_0, r_1, e_1, \ldots, r_L, e_L)$ is first serialized into a text sequence:

$$\text{str}(p) = e_0 \parallel r_1 \parallel e_1 \parallel \cdots \parallel r_L \parallel e_L \tag{3.8}$$

where entity IDs are mapped to readable Freebase names when available, and relation strings are normalized (e.g., underscore replacement). The query-side context is serialized as:

$$\text{query}(q, y_{<t}) = q \parallel \texttt{[SEP]} \parallel \text{str}(y_{<t}) \tag{3.9}$$

where $\text{str}(y_{<t})$ is the detokenised generation prefix.

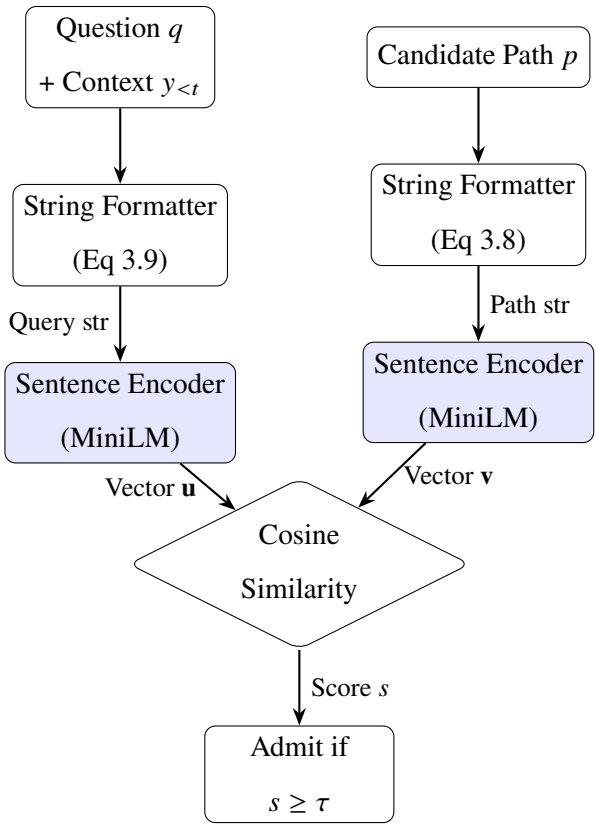

Figure 3.2 Semantic Relevance Scoring Mechanism.

Both $\text{str}(p)$ and $\text{query}(q, y_{<t})$ are encoded using `all-MiniLM-L6-v2` (Reimers and Gurevych, 2019) into 384-dimensional vectors. Relevance is computed as cosine similarity:

$$\text{score}(p,\ q,\ y_{<t}) = \cos\big(E(\text{str}(p))\,,\ \ E(\text{query}(q, y_{<t}))\big) \tag{3.10}$$

A candidate path is admitted into $\mathcal{W}_{\text{val}}^{\text{DCA}}(t)$ if and only if $\text{score}(p, q, y_{<t}) \geq \tau$, where τ is the admission threshold. This binary decision integrates the semantic score into the logit-masking framework.

3.5.2 Threshold Selection

The admission threshold τ is tuned on a separate 100-question WebQSP validation split. Selection prioritizes Condition P (low false negatives) while still maximizing trie reduction relative to GCR. We sweep $\tau \in [0.1, 0.6]$ with step size 0.05 and select the highest value that satisfies FNR < 0.05. Thresholds are tuned independently for v1 and v2, since v2 uses step-wise context updates and therefore follows a different pruning dynamic.

3.5.3 Encoder Caching

To reduce repeated computation during beam search, path embeddings $E(\text{str}(p))$ are cached. In v1, caching is done up front because the trie is fixed after construction. In v2, embeddings are cached lazily as new branches are expanded. In both cases, caching avoids redundant encoder calls for previously seen paths. By contrast, the query-context embedding $E(\text{query}(q, y_{<t}))$ must be recomputed whenever the generation prefix changes.

3.6 DCA-Trie v1: Static Semantic Filtering

3.6.1 Design Rationale

DCA-Trie v1 addresses the permissiveness problem while keeping the efficient static-trie design used in GCR. The key idea is to apply semantic filtering once, before decoding begins, using only the input question q. This is useful because many paths are structurally reachable but semantically unrelated to the question. Scoring paths against q during preprocessing removes weak candidates early and reduces the search space before autoregressive generation starts.

Methodologically, this changes the oracle from $f(\mathcal{G}, \mathcal{E}_q)$ to $f(\mathcal{G}, \mathcal{E}_q, q)$ without changing generation-time lookup mechanics. The decoder still queries a precompiled trie, so local lookup remains $O(d)$ as in GCR. Figure 3.3 summarises this offline filtering workflow.

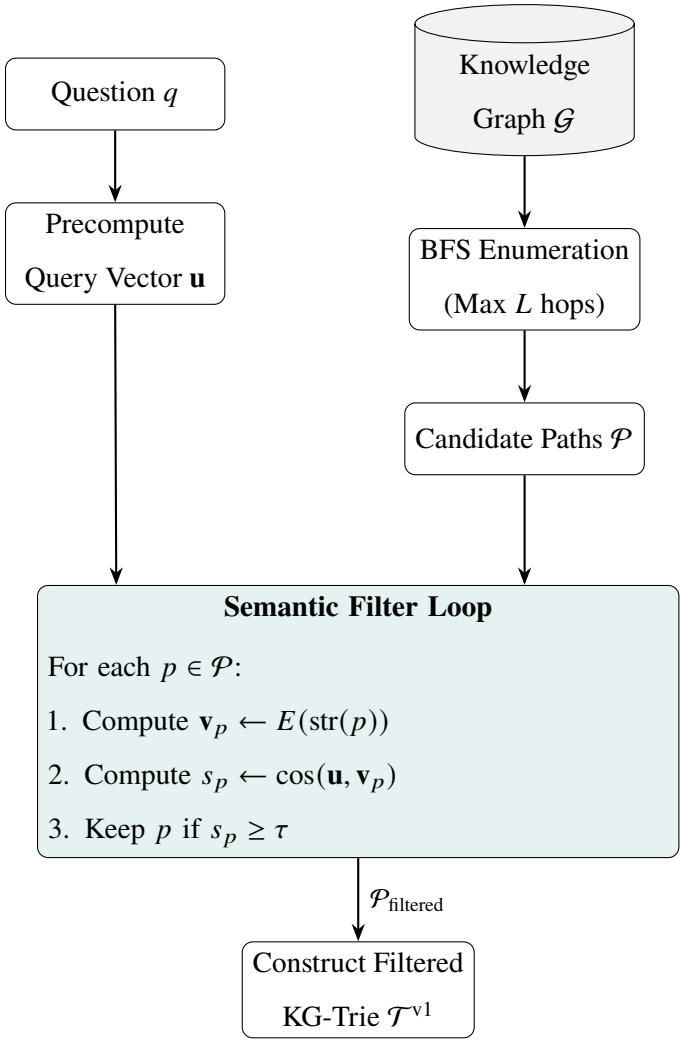

Figure 3.3 Static Semantic Filtering Pipeline for DCA-Trie v1.

3.6.2 Algorithm for DCA-Trie v1

Algorithm 1 runs in two separate phases: an offline filtering phase and an online generation phase. To explain how the mechanism works, the process unfolds in the following sequence:

1. **Precompute the Query State:** The system first embeds the raw question into a dense vector space, with an empty generation prefix at this stage.
2. **Enumerate the Structural Space:** The system uses standard breadth-first search (BFS) to map out every possible graph path starting from the question entities, going up to a maximum depth L. This is similar to the baseline GCR approach.
3. **Score and Prune:** This is where semantic intervention takes place. The system measures the cosine similarity between the question embedding and every path it has

enumerated. Any path that scores below the threshold τ is discarded as contextually irrelevant.

4. **Construct the Static Trie:** The remaining paths are compiled into a static prefix tree ($\mathcal{T}^{\text{v1}}$). This tree outlines the complete, semantically pruned valid search space.
5. **Autoregressive Decoding (Online):** During generation, the LLM queries the pre-built trie at each token step. The trie returns a binary mask that limits the model to output only tokens that follow the pre-approved paths.

This shows that semantic selectivity happens only once. By pruning the search space completely before generation starts, the decoding process stays fast, stable, and easily comparable to existing baselines.

Algorithm 1 DCA-Trie v1: Static Semantic Filtering at Construction Time

Require: Knowledge graph $\mathcal{G}$; question entities $\mathcal{E}_q$; question q; max hop depth L; admission threshold τ; sentence encoder E
Ensure: Semantically filtered KG-Trie $\mathcal{T}^{\text{v1}}$

Precompute query embedding: $\mathbf{u} \leftarrow E(\text{query}(q, \varepsilon))$ ▷ $y_{<t} = \varepsilon$ at construction time
Enumerate paths: $\mathcal{P} \leftarrow \text{BFS}(\mathcal{G}, \mathcal{E}_q, L)$ ▷ All paths within L hops, as in GCR
Score and filter:
for each path $p \in \mathcal{P}$ **do**
    $\mathbf{v}_p \leftarrow E(\text{str}(p))$ ▷ Precompute and cache path embedding
    $s_p \leftarrow \cos(\mathbf{u}, \mathbf{v}_p)$
    if $s_p \geq \tau$ **then**
        Add p to $\mathcal{P}_{\text{filtered}}$
    end if
end for
Construct filtered trie: $\mathcal{T}^{\text{v1}} \leftarrow \textsc{BuildTrie}(\mathcal{P}_{\text{filtered}})$
return $\mathcal{T}^{\text{v1}}$

At generation time (identical to GCR):
for each step t **do**
    $\mathcal{V}_t \leftarrow \textsc{TrieLookup}(\mathcal{T}^{\text{v1}}, y_{<t})$
    $\mathbf{m}_t[v] \leftarrow \mathbf{1}[v \in \mathcal{V}_t]$ for all v
    $\tilde{\alpha}_t \leftarrow \mathbf{m}_t \odot \text{LLMLogits}(y_{<t}, x)$
    $y_t \leftarrow$ beam-search-step$(\tilde{\alpha}_t)$
end for

Figure 3.4 DCA-Trie v1 algorithm.

3.6.3 Complexity Analysis

This section presents a complexity analysis of DCA-Trie v1 at the implementation level, based on the data structure assumptions used in this project. Building $\mathcal{T}^{\text{v1}}$ has an offline cost of $O(|\mathcal{P}| \cdot d_E)$, where $|\mathcal{P}|$ is the number of BFS-enumerated paths and $d_E = 384$ is the embedding dimension. This is a one-time preprocessing cost. During decoding, the lookup complexity is $O(d)$, which matches GCR, because accessing the trie uses the same constant-time child lookup structure. Memory usage scales as $O(|\mathcal{P}_{\text{filtered}}| \cdot L)$, which is usually much smaller than GCR's $O(|\mathcal{P}| \cdot L)$ because semantic filtering removes irrelevant paths.

3.6.4 Faithfulness Guarantee

DCA-Trie v1 maintains structural faithfulness by design. Every path kept in $\mathcal{P}_{\text{filtered}}$ comes from $\text{BFS}(\mathcal{G}, \mathcal{E}_q, L)$, ensuring that each triplet (e_{i-1}, r_i, e_i) remains a valid graph edge. Semantic filtering only removes paths and does not add new tokens or edges. Therefore, any token passed through $\mathcal{T}^{\text{v1}}$ stays structurally grounded, meeting Condition F when the token-mask construction and tokenizer alignment are correct.

3.7 DCA-Trie v2: Step-Wise Dynamic Expansion

3.7.1 Design Rationale

DCA-Trie v2 implements the full dynamic oracle in Equation (3.2) by conditioning constraints on the evolving prefix $y_{<t}$. Unlike v1, it does not assume that question-level intent alone is sufficient throughout decoding. As generation progresses, $y_{<t}$ reveals the path decisions already made, so the relevant neighbourhood can change step by step. For example, after selecting *Elvis_Presley* → *starred_in* → *Blue_Hawaii*, plausible next expansions should focus on *Blue_Hawaii* rather than the initial entity set.

Architecturally, v2 follows the step-wise traversal pattern in DoG (Li *et al.*, 2025) but adds semantic gating to each local expansion. After each committed entity e_t, the trie expands only to structurally valid neighbours that also satisfy the threshold τ:

$$\mathcal{T}_{t+1} = \mathcal{T}_t \cup \left\{(e_t, r, e') : (e_t, r, e') \in \mathcal{T}_{\mathcal{G}} \ \wedge \ \text{score}(e', q, y_{<t}) \geq \tau\right\} \tag{3.11}$$

where $\mathcal{T}_{\mathcal{G}}$ is the set of graph triplets. The core difference from permissive dynamic expansion is therefore not when expansion happens, but how candidates are admitted: every step remains graph-valid and semantically filtered. The full workflow is shown in Figure 3.5.

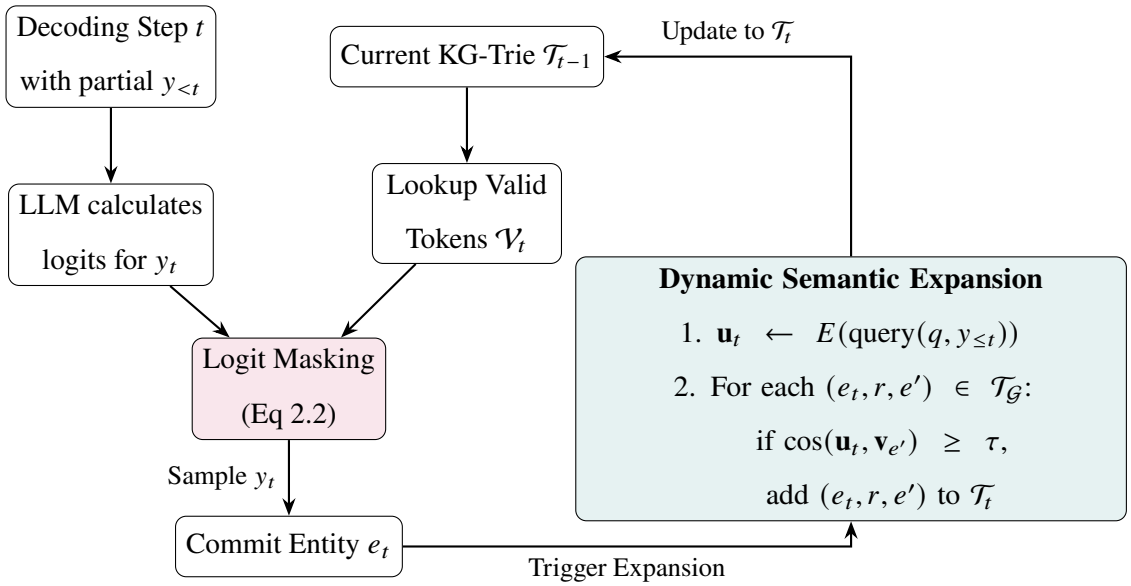

Figure 3.5 Step-Wise Dynamic Expansion Workflow for DCA-Trie v2.

3.7.2 Algorithm for DCA-Trie v2

Unlike the static approach in v1, Algorithm 2 combines generation and semantic filtering. To manage how the search space changes, the process follows these steps:

1. **Initialize a Minimal State:** The system starts with a basic trie that includes only the root question entities and a baseline query embedding.
2. **Generate under Current Constraints:** At each step t, the LLM produces a new token that is strictly limited by the current physical boundaries of the trie.
3. **Check for Entity Commitment:** The algorithm branches based on what the LLM just produced. If it generates an intermediate relation token, the trie stays the same and decoding continues. If the LLM finishes generating an entity token, it triggers an expansion phase.
4. **Update the Context:** When an entity is committed, the system recalculates the query embedding. This new embedding merges the original question with the specific

reasoning path the LLM just created, making the context very specific to the current state.

5. **Dynamically Expand:** The system pulls all structurally valid neighbours of the newly committed entity from the master knowledge graph. It scores these candidates against the updated context embedding and discards the irrelevant ones. The remaining candidates are then added to the trie to enable valid future generation.

This means we have a highly adaptable oracle. Instead of forcing the LLM to pick from a large, pre-calculated network of paths, the system builds the network just one step ahead of the model's current reasoning state, following its path while keeping the graph valid.

Algorithm 2 DCA-Trie v2: Step-Wise Semantic Expansion

Require: Knowledge graph $\mathcal{G}$; question entities $\mathcal{E}_q$; question q; admission threshold τ; sentence encoder E; LLM with vocabulary $\mathcal{V}$
Ensure: Generated reasoning chain $y_1 \dots y_T$

Initialize trie with entity nodes for $\mathcal{E}_q$: $\mathcal{T}_0 \leftarrow \{e_q : e_q \in \mathcal{E}_q\}$
Precompute query embedding: $\mathbf{u}_0 \leftarrow E(\text{query}(q, \varepsilon))$
for each step $t = 1$ **to** T **do**
    $\mathcal{V}_t \leftarrow \textsc{TrieLookup}(\mathcal{T}_{t-1}, y_{<t})$ ▷ Valid tokens from current trie
    $\mathbf{m}_t[v] \leftarrow \mathbf{1}[v \in \mathcal{V}_t]$ for all v
    $\tilde{\alpha}_t \leftarrow \mathbf{m}_t \odot \text{LLMLogits}(y_{<t}, x)$
    $y_t \leftarrow$ beam-search-step$(\tilde{\alpha}_t)$
    if y_t is an *entity* token e_t **then** ▷ Expansion triggered at entity commits
        Update query embedding: $\mathbf{u}_t \leftarrow E(\text{query}(q, y_{\le t}))$
        Expand trie:
        for each $(e_t, r, e') \in \mathcal{T}_{\mathcal{G}}$ **do**
            $\mathbf{v}_{e'} \leftarrow E(\text{str}((e_t, r, e')))$ ▷ Cache if previously computed
            $s \leftarrow \cos(\mathbf{u}_t, \mathbf{v}_{e'})$
            if $s \ge \tau$ **then**
                Add (e_t, r, e') to $\mathcal{T}_t$
            end if
        end for
    else
        $\mathcal{T}_t \leftarrow \mathcal{T}_{t-1}$ ▷ No expansion at relation token steps
    end if
end for
return $y_1 \dots y_T$

Figure 3.6 DCA-Trie v2 algorithm.

3.7.3 Complexity Analysis

This section presents a complexity analysis of DCA-Trie v2 at the implementation level under the same assumptions. In v2, the expansion cost only happens when an entity token is committed, with a complexity of $O(|\mathcal{N}_{e_t}| \cdot d_E)$, where $|\mathcal{N}_{e_t}|$ is the neighborhood size of e_t. In practice, semantic filtering reduces this to about $O(|\mathcal{N}_{e_t}^{\text{filtered}}| \cdot d_E)$ with $|\mathcal{N}_{e_t}^{\text{filtered}}| \ll d$ for most nodes.

Relation-token steps do not trigger expansion, so their cost stays similar to that of standard constrained decoding. Caching further lowers overhead by preventing the repeated encoding of candidates that have already been seen.

3.7.4 Faithfulness Guarantee

As in v1, v2 maintains structural faithfulness by design. Every expansion candidate must meet the requirement $(e_t, r, e') \in \mathcal{T}_{\mathcal{G}}$ in Equation (3.11). Semantic scoring only prunes this structurally valid set; it does not introduce new graph transitions. Therefore, all accepted tokens remain graph-based, and Condition F holds true under proper token-mask construction and tokenizer alignment.

3.8 Baseline Systems

Four system configurations are evaluated to assess the proposed methodology:

CoT Baseline: The same Llama-3.1-8B model (Dubey *et al.*, 2024) used in GCR, run without any KG constraint oracle and prompted with standard chain-of-thought reasoning. This provides the unconstrained reference point.

GCR: The original Graph-Constrained Reasoning framework (Luo *et al.*, 2025a) with static KG-Trie constraints $\mathcal{W}_{\text{val}}(t) = f(\mathcal{G}, \mathcal{E}_q)$. This is the primary constrained baseline. Reproduction targets WebQSP Hits@1 within 1–2% of the reported 92.6%.

DCA-Trie v1: The static semantic filtering variant described in Section 3.6, implemented as a preprocessing step in the GCR pipeline with validation-calibrated threshold τ.

DCA-Trie v2: The dynamic step-wise expansion variant described in Section 3.7, integrated into the decoding process.

To ensure fair comparison, all settings use the same Llama-3.1-8B backbone, the same entity-linking pipeline for $\mathcal{E}_q$, and matched decoding settings (beam width $k = 5$ for constrained systems). This isolates differences to the constraint-oracle design rather than model or preprocessing variation.

3.9 Evaluation Protocol

3.9.1 Datasets

Evaluation is conducted on two standard Freebase-based KGQA datasets (Bollacker *et al.*, 2008):

WebQSP (Yih *et al.*, 2016): 4,737 questions, mostly requiring 1–2 hops. The standard test set is used for evaluation. A separate disjoint set of 100 questions is used only for threshold calibration.

ComplexWebQuestions (CWQ) (Talmor and Berant, 2018): 34,689 questions with higher compositional complexity and hop depths up to 4. This dataset is used in full for evaluation with no overlap with the calibration split.

3.9.2 Evaluation Metrics

Five metrics are used across both datasets:

1. **Hits@1:** The proportion of examples where the top predicted answer matches the gold entity.
2. **F1:** Entity-level overlap score for settings with multiple acceptable answers, following common KGQA evaluation practice (Lan *et al.*, 2022a).
3. **Structural Faithfulness Rate:** The fraction of generated paths where all triplets are valid in the underlying KG.
4. **Semantic Irrelevance Ratio (SIR):** Computed using Equations (3.6) and (3.7) with a fixed reference threshold $\tau_{\text{ref}} = 0.3$.
5. **Average Trie Size Per Step:** The mean size of the valid token set $|\mathcal{V}_t|$ across decoding steps, used as a direct efficiency indicator of constraint selectivity.

3.9.3 Hop-Depth Stratification

All major metrics (except binary structural faithfulness) are also reported by hop depth (1, 2, 3, and 4), consistent with prior KGQA analysis (Lan *et al.*, 2022a). This is important because permissiveness effects usually increase with hop depth; reporting only aggregate results can hide this behavior.

3.9.4 Experimental Configuration

All experiments are run on a single NVIDIA A100 (40GB). Llama-3.1-8B inference uses 16-bit precision through HuggingFace Transformers (Wolf *et al.*, 2020). Constrained systems use beam width $k = 5$, while the unconstrained CoT baseline uses $k = 1$ to match the original GCR setup. The `all-MiniLM-L6-v2` encoder runs through 'sentence-transformers' (Reimers and Gurevych, 2019) on CPU to avoid GPU contention with LLM decoding. Maximum generation length is set to $3L + 2$, with $L = 2$ for WebQSP and $L = 4$ for CWQ.

3.10 Scope Boundaries and Anticipated Limitations

This study has clear scope boundaries that frame the interpretation of the Chapter 4 results:

Single KG and domain: Experiments are limited to Freebase-based KGQA. Generalization to other KGs (e.g., Wikidata and ConceptNet) remains future work.

No model fine-tuning: The Llama-3.1-8B backbone is not fine-tuned. Observed improvements reflect changes in constraint-oracle design rather than adaptation of parameters.

Empirical faithfulness evidence: Condition F is assessed empirically by checking generated paths against KG triplets, following established practice in prior work (Luo *et al.*, 2025a; Li *et al.*, 2025). This evidence is interpreted under the same implementation assumptions regarding valid-token masking and tokenizer alignment.

Threshold as a hyperparameter: The selected τ is tuned on a validation subset and may not transfer well across datasets.

False negative risk: Even under Condition P, semantic filtering can remove valid paths, particularly when lexical form and graph semantics diverge. This trade-off is explicitly analyzed in Chapter 4.

Latency: Semantic scoring adds extra compute overhead compared to GCR. Wall-clock latency is reported, but production deployment optimization is outside the scope of this thesis.

REFERENCES

Agrawal, G., Kumarage, T., Alghamdi, Z., and Liu, H. (2024), "Mindful-RAG: A Study of Points of Failure in Retrieval Augmented Generation", *2024 2nd International Conference on Foundation and Large Language Models (FLLM)*, pp. 607–611.

Anderson, P., Fernando, B., Johnson, M., and Gould, S. (2017), "Guided Open Vocabulary Image Captioning with Constrained Beam Search", *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, pp. 936–945.

Asai, A., Hashimoto, K., Hajishirzi, H., Socher, R., and Xiong, C. (2020), "Learning to Retrieve Reasoning Paths over Wikipedia Graph for Question Answering", *International Conference on Learning Representations*.

Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. (2024), "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection", *International Conference on Learning Representations*.

Bast, H. and Haussmann, E. (2015), "More Accurate Question Answering on Freebase", *Proceedings of the 24th ACM International on Conference on Information and Knowledge Management*.

Berant, J., Chou, A., Frostig, R., and Liang, P. (2013), "Semantic Parsing on Freebase from Question-Answer Pairs", *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*.

Bollacker, K., Evans, C., Paritosh, P., Sturge, T., and Taylor, J. (2008), "Freebase: A Collaboratively Created Graph Database for Structuring Human Knowledge", *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, pp. 1247–1250.

Bosma, M., Chi, E., Ichter, B., Le, Q. V., Schuurmans, D., Wang, X., Wei, J., Xia, F., and Zhou, D. (2022), "Chain-Of-Thought Prompting Elicits Reasoning in Large Language Models", *Advances in Neural Information Processing Systems 35*, NeurIPS 2022, Neural Information Processing Systems Foundation, Inc. (NeurIPS), pp. 24824–24837.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T.,

Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. (2020), "Language Models are Few-Shot Learners", *Advances in Neural Information Processing Systems*, vol. 33, Curran Associates, Inc., pp. 1877–1901.

Cao, Y., Griffiths, T., Narasimhan, K., Shafran, I., Yao, S., Yu, D., and Zhao, J. (2023a), "Tree of Thoughts: Deliberate Problem Solving with Large Language Models", *Advances in Neural Information Processing Systems 36*, NeurIPS 2023, Neural Information Processing Systems Foundation, Inc. (NeurIPS), pp. 11809–11822.

Cao, Y., Griffiths, T., Narasimhan, K., Shafran, I., Yao, S., Yu, D., and Zhao, J. (2023b), "Tree of Thoughts: Deliberate Problem Solving with Large Language Models", *Advances in Neural Information Processing Systems 36*, NeurIPS 2023, Neural Information Processing Systems Foundation, Inc. (NeurIPS), pp. 11809–11822.

Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., and Liu, Z. (2024), "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation", *arXiv preprint arXiv:2402.03216*.

Das, R., Dhuliawala, S., Zaheer, M., Vilnis, L., Durugkar, I., Krishnamurthy, A., Smola, A., and McCallum, A. (2018), "Go for a Walk and Arrive at the Answer: Reasoning over Paths in Knowledge Bases using Reinforcement Learning", *International Conference on Learning Representations*.

De Cao, N., Izacard, G., Riedel, S., and Petroni, F. (2021), "Autoregressive Entity Retrieval", *arXiv preprint arXiv:2010.00904*, ICLR 2021.

Deutsch, D., Upadhyay, S., and Roth, D. (2019), "A General-Purpose Algorithm for Constrained Sequential Inference", *Proceedings of the 23rd Conference on Computational Natural Language Learning (CoNLL)*, Association for Computational Linguistics, pp. 482–492.

Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., and Fan, A. (2024), "The Llama 3 Herd of Models", *arXiv preprint arXiv:2407.21783*.

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., and Larson, J. (2024), "From Local to Global: A Graph RAG Approach to Query-Focused Summarization", *arXiv preprint arXiv:2404.16130*.

Fredkin, E. (1960), "Trie Memory", *Communications of the ACM*, Vol. 3, No. 9, pp. 490–499.

Gao, L., Dai, Z., Pasupat, P., Chen, A., Chaganty, A. T., Fan, Y., Zhao, V., Lao, N., Lee, H., Juan, D.-C., and Guu, K. (July 2023), "RARR: Researching and Revising What Language Models Say, Using Language Models", *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ed. by A. Rogers, J. Boyd-Graber, and N. Okazaki, Toronto, Canada: Association for Computational Linguistics, pp. 16477–16508.

Geng, S., Josifoski, M., Peyrard, M., and West, R. (2023), "Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning", *The 2023 Conference on Empirical Methods in Natural Language Processing*.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., and Vaughan (2024), "The llama 3 herd of models", *arXiv preprint arXiv:2407.21783*.

Han, X., Cao, S., Lv, X., Lin, Y., Liu, Z., Sun, M., and Li, J. (2018), "OpenKE: An Open Toolkit for Knowledge Embedding", *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, Association for Computational Linguistics, pp. 139–144.

He, G., Lan, Y., Jiang, J., Zhao, W. X., and Wen, J.-R. (2021a), "Improving Multi-Hop Knowledge Base Question Answering by Learning Intermediate Supervision Signals", *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*, pp. 553–561.

He, G., Lan, Y., Jiang, J., Zhao, W. X., and Wen, J.-R. (2021b), "Improving Multi-Hop Knowledge Base Question Answering by Learning Intermediate Supervision Signals", *Proceedings of the 14th ACM International Conference on Web Search and Data Mining (WSDM)*, ACM, pp. 553–561.

He, G., Lan, Y., Jiang, J., Zhao, W. X., and Wen, J.-R. (2021c), "Improving Multi-Hop Knowledge Base Question Answering by Learning Intermediate Supervision Signals", *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*, pp. 553–561.

He, H., Zhang, H., Yang, D., and Roth, D. (2022), "Rethinking with Retrieval: Faithful Reasoning starting from Knowledge Graphs", *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 5461–5475.

He, X., Tian, Y., Sun, Y., Chawla, N. V., Laurent, T., LeCun, Y., Bresson, X., and Hooi, B. (2024), "G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering", *arXiv preprint arXiv:2402.07630*.

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Las Casas, D. de, Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., Driessche, G. van den, Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Vinyals, O., Rae, J. W., and Sifre, L. (2022), "Training compute-optimal large language models", *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS '22, New Orleans, LA, USA: Curran Associates Inc.

Hokamp, C. and Liu, Q. (2017), "Lexically Constrained Decoding for Sequence Generation Using Grid Beam Search", *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, pp. 1535–1546.

Holtzman, A., Buys, J., Du, L., Forbes, M., and Choi, Y. (2020), "The Curious Case of Neural Text Degeneration", *International Conference on Learning Representations*.

Huang, J. and Chang, K. C.-C. (2023), "Towards Reasoning in Large Language Models: A Survey", *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 1049–1065.

Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., and Zhou, D. (2024), "Large Language Models Cannot Self-Correct Reasoning Yet", *The Twelfth International Conference on Learning Representations*.

Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., and Liu, T. (2023), "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions", *arXiv preprint arXiv:2311.05232*.

Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., and Liu, T. (Jan. 2025), "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions", *ACM Trans. Inf. Syst.*, Vol. 43, No. 2.

Izacard, G. and Grave, E. (2021), "Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering", *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, pp. 874–880.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., and Fung, P. (Mar. 2023), "Survey of Hallucination in Natural Language Generation", *ACM Comput. Surv.*, Vol. 55, No. 12.

Jiang, J., Zhou, K., Dong, Z., Ye, K., Zhao, X., and Wen, J.-R. (2023a), "StructGPT: A General Framework for Large Language Model to Reason over Structured Data", *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, pp. 9237–9251.

Jiang, J., Zhou, K., Zhao, X., and Wen, J.-R. (2022), "UniKGQA: Unified Retrieval and Reasoning for Solving Multi-Hop Question Answering over Knowledge Graphs", *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2022)*.

Jiang, Z., Xu, F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., Yang, Y., Callan, J., and Neubig, G. (2023b), "Active Retrieval Augmented Generation", *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, pp. 7969–7992.

Jin, B., Liu, G., Han, C., Jiang, M., Ji, H., and Han, J. (2024), "Graph Chain-of-Thought: Augmenting Large Language Models by Reasoning on Graphs", *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 163–184.

Kandpal, N., Deng, H., Roberts, A., Wallace, E., and Raffel, C. (2023), "Large Language Models Struggle to Learn Long-Tail Knowledge", *Proceedings of the 40th International Conference on Machine Learning (ICML)*, PMLR, pp. 15696–15707.

Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., and Yih, W.-t. (2020), "Dense Passage Retrieval for Open-Domain Question Answering", *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, pp. 6769–6781.

Khot, T., Trivedi, H., Finlayson, M., Fu, Y., Richardson, K., Clark, P., and Sabharwal, A. (2023), "Decomposed Prompting: A Modular Approach for Solving Complex Tasks", *The Eleventh International Conference on Learning Representations*.

Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y. (2022), "Large Language Models are Zero-Shot Reasoners", *Advances in Neural Information Processing Systems*, vol. 35.

Lan, Y., He, G., Jiang, J., Jiang, J., Zhao, W. X., and Wen, J.-R. (2022a), "Complex Knowledge Base Question Answering: A Survey", *arXiv preprint arXiv:2108.06688*.

Lan, Y., He, G., Jiang, J., Jiang, J., Zhao, W. X., and Wen, J.-R. (2022b), "Complex Knowledge Base Question Answering: A Survey", *arXiv preprint arXiv:2108.06688*.

Lan, Y. and Jiang, J. (2020), "Query Graph Generation for Answering Multi-hop Complex Questions from Knowledge Bases", *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 969–974.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020), "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", *Advances in Neural Information Processing Systems*, vol. 33, Curran Associates, Inc., pp. 9459–9474.

Li, K., Zhang, T., Wu, X., Luo, H., Glass, J. R., and Meng, H. M. (2025), "Decoding on Graphs: Faithful and Sound Reasoning on Knowledge Graphs through Generation of Well-Formed Chains", *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, pp. 24349–24364.

Lin, S., Hilton, J., and Evans, O. (2022), "TruthfulQA: Measuring How Models Mimic Human Falsehoods", *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, Association for Computational Linguistics, pp. 3214–3252.

Lin, Z., Yang, H., and Zhang, M. (2022), "Rethinking Knowledge Graph Evaluation Under the Open-World Assumption", *Advances in Neural Information Processing Systems 35*, NeurIPS 2022, Neural Information Processing Systems Foundation, Inc. (NeurIPS), pp. 8374–8385.

Lowerre, B. T. (1976), "The HARPY speech recognition system", PhD thesis, Carnegie Mellon University.

Lu, X., West, P., Zellers, R., Le Bras, R., Bhagavatula, C., and Choi, Y. (2021), "NeuroLogic Decoding: (Un)supervised Neural Text Generation with Predicate Logic Constraints", *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, Association for Computational Linguistics, pp. 4288–4299.

Luo, L., Li, Y.-F., Haffari, G., and Pan, S. (2024), "Reasoning on Graphs: Faithful and Interpretable Large Language Model Reasoning", *International Conference on Learning Representations*.

Luo, L., Zhao, Z., Haffari, G., Li, Y.-F., Gong, C., and Pan, S. (2025a), "Graph-constrained Reasoning: Faithful Reasoning on Knowledge Graphs with Large Language Models", *Proceedings of the 42nd International Conference on Machine Learning*, vol. 267, Proceedings of Machine Learning Research, PMLR, pp. 41540–41565.

Luo, L., Zhao, Z., Haffari, G., Phung, D., Gong, C., and Pan, S. (2025b), "GFM-RAG: Graph Foundation Model for Retrieval Augmented Generation", *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Mallen, A., Asai, A., Zhong, V., Das, R., Khashabi, D., and Hajishirzi, H. (2023), "When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories", *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, Association for Computational Linguistics, pp. 9802–9822.

Markowitz, E., Ramakrishna, A., Dhamala, J., Mehrabi, N., Peris, C., Gupta, R., Chang, K.-W., and Galstyan, A. (Aug. 2024), "Tree-of-Traversals: A Zero-Shot Reasoning Algorithm for Augmenting Black-box Language Models with Knowledge Graphs", *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ed. by L.-W. Ku, A. Martins, and V. Srikumar, Bangkok, Thailand: Association for Computational Linguistics, pp. 12302–12319.

Mavromatis, C. and Karypis, G. (2022), "ReaRev: Adaptive Reasoning for Question Answering over Knowledge Graphs", *Findings of the Association for Computational Linguistics: EMNLP 2022*, Association for Computational Linguistics, pp. 4447–4458.

Mavromatis, C. and Karypis, G. (2025), "GNN-RAG: Graph Neural Retrieval for Efficient Large Language Model Reasoning on Knowledge Graphs", *Findings of the Association for Computational Linguistics: ACL 2025*, Association for Computational Linguistics, pp. 16682–16699.

Miller, A., Fisch, A., Dodge, J., Karimi, A.-H., Bordes, A., and Weston, J. (2016), "Key-value memory networks for directly reading documents", *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*.

Nakano, R., Hilton, J., Balaji, S., Wu, J., Ouyang, L., Kim, C., Hesse, C., Jain, S., Kosaraju, V., Saunders, W., Jiang, X., Cobbe, K., Eloundou, T., Krueger, G., Button, K., Knight, M., Chess, B., and Schulman, J. (2022), "WebGPT: Browser-assisted question-answering with human feedback".

OpenAI (2024), "GPT-4 Technical Report".

Perez, E., Ringer, S., Lukošiūtė, K., Nguyen, K., Chen, E., Heiner, S., Pettit, C., Olsson, C., Kundu, S., Kadavath, S., Jones, A., Chen, A., Mann, B., Israel, B., Bowman, S. R., Clark, J., Ganguli, D., Askell, A., and Hernandez, D. (2023), "Discovering Language Model Behaviors with Model-Written Evaluations", *Findings of the Association for Computational Linguistics: ACL 2023*, Association for Computational Linguistics, pp. 13387–13434.

Poesia, G., Polozov, O., Le, V., Tiwari, A., Soares, G., Meek, C., and Gulwani, S. (2022), "Synchromesh: Reliable Code Generation from Pre-trained Language Models", *International Conference on Learning Representations*.

Reddy, S., Lapata, M., and Steedman, M. (2014), "Large-scale semantic parsing without question-answer pairs", *Transactions of the Association for Computational Linguistics*.

Reimers, N. and Gurevych, I. (2019), "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks", *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, pp. 3982–3992.

Saxena, A., Chakrabarti, S., and Talukdar, P. (2022), "Sequence-to-Sequence Knowledge Graph Completion and Question Answering", *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, Association for Computational Linguistics, pp. 2814–2828.

Saxena, A., Tripathi, A., and Talukdar, P. (2020), "Improving Multi-hop Question Answering over Knowledge Graphs using Knowledge Base Embeddings", *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*.

Scholak, T., Schucher, N., and Bahdanau, D. (2021), "PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models", *arXiv preprint arXiv:2109.05093*, EMNLP 2021.

Sen, P., Mavadia, S., and Saffari, A. (June 2023), "Knowledge Graph-augmented Language Models for Complex Question Answering", *Proceedings of the 1st Workshop on Natural Language Reasoning and Structured Explanations (NLRSE)*, ed. by B. Dalvi Mishra, G. Durrett, P. Jansen, D. Neves Ribeiro, and J. Wei, Toronto, Canada: Association for Computational Linguistics, pp. 1–8.

Shi, J., Cao, S., Hou, L., Li, J., and Zhang, H. (2021), "transferNet: An Effective and Transparent Framework for Multi-Hop Question Answering over Relation Graph", *Proceedings of*

the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP), Association for Computational Linguistics, pp. 4149–4158.

Sun, H., Dhingra, B., Zaheer, M., Mazaitis, K., Salakhutdinov, R., and Cohen, W. (2018), "Open Domain Question Answering Using Early Fusion of Knowledge Bases and Text", *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 4231–4242.

Sun, H., Dhingra, B., Zaheer, M., Mazaitis, K., Salakhutdinov, R., and Cohen, W. W. (2019), "PullNet: Open Domain Question Answering with Iterative Retrieval on Knowledge Bases and Text", *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, pp. 2380–2390.

Sun, J., Xu, C., Tang, L., Wang, S., Lin, C., Gong, Y., Ni, L. M., Shum, H.-Y., and Guo, J. (2024), "Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph", *International Conference on Learning Representations*.

Talmor, A. and Berant, J. (2018), "The Web as a Knowledge-Base for Answering Complex Questions", *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, Association for Computational Linguistics, pp. 641–651.

Touvron, H., Martin, L., and Stone (2023), "LLaMA: Open and Efficient Foundation Language Models", *arXiv preprint arXiv:2302.13971*.

Trivedi, H., Balasubramanian, N., Khot, T., and Sabharwal, A. (2023), "Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions", *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, Association for Computational Linguistics, pp. 10014–10037.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017), "Attention Is All You Need", *arXiv preprint arXiv:1706.03762*, NeurIPS 2017.

Wang, K., Duan, F., Wang, S., Li, P., Xian, Y., Yin, C., Rong, W., and Xiong, Z. (2023a), "Knowledge-Driven CoT: Exploring Faithful Reasoning in LLMs for Knowledge-intensive Question Answering".

Wang, L., Xu, W., Lan, Y., Hu, Z., Lan, Y., Lee, R. K.-W., and Lim, E.-P. (2023b), "Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language

Models", *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 2609–2634.

Wang, X., Gao, T., Zhu, Z., Zhang, Z., Liu, Z., Li, J., and Tang, J. (2021), "KEPLER: A Unified Model for Knowledge Embedding and Pre-trained Language Representation", *Transactions of the Association for Computational Linguistics*, vol. 9, MIT Press, pp. 176–194.

Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., and Zhou, D. (2023c), "Self-Consistency Improves Chain of Thought Reasoning in Language Models", *The Eleventh International Conference on Learning Representations*.

Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., and Zhou, D. (2023d), "Self-Consistency Improves Chain of Thought Reasoning in Language Models", *International Conference on Learning Representations*.

Willard, B. T. and Louf, R. (2023), "Efficient Guided Generation for Large Language Models".

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., Platen, P. von, Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., and Rush, A. (Oct. 2020), "Transformers: State-of-the-Art Natural Language Processing", *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, ed. by Q. Liu and D. Schlangen, Online: Association for Computational Linguistics, pp. 38–45.

Xiong, L., Xiong, C., Li, Y., Tang, K.-F., Liu, J., Bennett, P., Ahmed, J., and Overwijk, A. (2021), "Approximate Nearest Neighbor Negative Contrastive Estimation for Dense Text Retrieval", *International Conference on Learning Representations*.

Xu, K., Lai, Y., Feng, Y., and Zhao, D. (2019), "Enhancing key-value memory neural networks for knowledge based question answering", *Proceedings of NAACL-HLT*.

Xu, Z., Jain, S., and Kankanhalli, M. (2025), "Hallucination is Inevitable: An Innate Limitation of Large Language Models".

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023), "ReAct: Synergizing Reasoning and Acting in Language Models", *International Conference on Learning Representations*.

Yasunaga, M., Ren, H., Bosselut, A., Liang, P., and Leskovec, J. (2021), "QA-GNN: Reasoning with Language Models and Knowledge Graphs for Question Answering", *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational*

Linguistics: Human Language Technologies (NAACL-HLT), Association for Computational Linguistics, pp. 535–546.

Yih, W.-t., Chang, M.-W., He, X., and Gao, J. (2015), "Semantic Parsing via Staged Query Graph Generation: Question Answering on Freebase", *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics*.

Yih, W.-t., Richardson, M., Meek, C., Chang, M.-W., and Suh, J. (2016), "The Value of Semantic Parse Labeling for Knowledge Base Question Answering", *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, Association for Computational Linguistics, pp. 201–206.

Yu, D., Chen, C., Onoe, Y., and Qi, P. (2023), "DECAF: Joint Decoding of Answers and Logical Forms for Question Answering over Knowledge Bases", *The Eleventh International Conference on Learning Representations*.

Yuan, B., Deng, H., Liu, X., and Zhang, M. (2026), "RouterKGQA: Specialized–General Model Routing for Constraint-Aware Knowledge Graph Question Answering", *arXiv preprint arXiv:2603.20017*.

Zhang, H., Dang, M., Peng, N., and Van den Broeck, G. (2023), "Tractable Control for Autoregressive Language Generation", *International Conference on Machine Learning*.

Zhang, J., Zhang, X., Yu, J., Tang, J., Tang, J., Li, C., and Chen, H. (2022), "Subgraph Retrieval Enhanced Model for Multi-hop Knowledge Base Question Answering", *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, pp. 5773–5784.

Zhang, Y., Dai, H., Kozareva, Z., Smola, A. J., and Song, L. (2018), "Variational reasoning for question answering with knowledge graph", *Proceedings of AAAI*.

Zhuang, A., Zhang, G., Zheng, T., Du, X., Wang, J., Ren, W., Huang, W., Fu, J., Yue, X., and Chen, W. (2024), "StructLM: Towards Building Generalist Models for Structured Knowledge Grounding", *First Conference on Language Modeling*.